MiniUI

Cross Platform User Interface Library

Table of Contents

Introduction 5
- What is MiniUI 5
- Why another UI system? 5
- How is MiniUI different? 5
- The current development process 6
- Code based user interfaces 8
- Current XML based UI frameworks 9
- Scripting 9

Hardware Acceleration 11
- OpenGL 11
- SVG 12
- Expansion 12

Case Study: The confirmation dialog 13

Three steps to a new UI 14
- The skin file 14
- The layout file 14
- The Script file 15

Breaking down the skin file 17
- skin/include 17
- skin/widgets/widget 17
- .../widget/layout 18
- .../widget/layout/section 18
- .../widget/layout/section/source 19
- .../widget/layout/section/destination 19
- .../widget/children 19
- .../widget/children/area 20
- skin/fonts 21
- .../fonts/font 21

Case Study: The skin file 22

Breaking down the layout file 27
- interface/screen 27

Case Study: The layout file 28

Breaking down the script file 30
- Screen space versus widget space 30
- The widget class 32
  - Widget:__init() 33
  - Widget:OnLoad(skinElement, layoutElement) 34
  - Widget:OnLayout() 34
  - Widget:OnMouseOver() 34
  - Widget:OnMouseOut() 34
  - Widget:OnMouseHover(x, y) 34
  - Widget:OnMouseMove(x, y) 35
  - Widget:OnMouseDown(x, y) 35
  - Widget:OnMouseUp(x, y) 35
  - Widget:Call(function, object) 35
  - Widget:Update(timestep) 36
  - Widget:Fire(event, object) 36

Widget:GetWidgetByID(id) 36
Widget:GetChildWidget(area, widget) 36
Widget:GetChildWidgetCount(area) 37
Widget:GetGraphicalRect(id) 37
Widget:UpdateRenderable() 37
Widget:AddChild(area, widget) 38
Widget:angle 38
Widget:x and Widget:y 38
Widget:widget and Widget:height 38
Widget:opacity 38
Widget:id 39
GraphicalRect class 39
GraphicalRect:id 39
GraphicalRect:isVisible 39
MiniUI class 40
MiniUI:CreateWidget (widgetID) [static] 40
xpath class 40
xpath:ToElement(xmlElement, path) [static] 40
xpath:ToString(xmlElement, path) [static] 40
xpath:ToNumber(xmlElement, path) [static] 41
Breaking down animations 42
Animator class 43
Animator( ) [static] 44
Animator:Run(duration) 44
Animator:Stop() 44
Animator:Add(animatable) 45
Animate class 45
Animate(object, attribute, animation, end, duration) [static] 45
Animate:Run(duration) 46
Animate:Stop() 46
Animate:Add(animatable) 46
Delay class 47
Delay (duration) [static] 47
Delay:Run(duration) 47
Delay:Stop() 48
Delay:Add(animatable) 48
EventNotify class 48
EventNotify (object, event) [static] 49
EventNotify:Run(duration) 49
EventNotify:Stop() 49
EventNotify:Add(animatable) 49
MiniUI class 50
MiniUI:FindWidget(id) [static] 50
Breaking down the C++ API 54
IArchive class 54
string IArchive::GetArchiveName( ) [overridden] 54
istream* IArchive::LoadFile(string filename) [overridden] 55
bool IArchive::FileExists (string filename) [overridden] 55
Mouse class 55
UIManager class 56
UIManager::UIManager( ) 56

bool UIManager::Initialize( ) 56
bool UIManager::LoadUIArchive (IArchive *) 57
bool UIManager::LoadScreens (string filename, IArchive *) 57
bool UIManager::Update (int timestep, Mouse*) 57
void UIManager::SetCurrentScreen (string screenName) 57
Screen* UIManager::GetCurrentScreen ( ) 57
void UIManager::FireEvent (Widget* w, string name, object o) 58
Screen class 58
Widget* Screen::FindWidget(string id) 58
The Event System 59
Event Class 59
Event::Event( ) 60
Event::~Event( ) 60
void Event::AddListener (Functor* functor) 60
void Event::RemoveListener (Functor* functor) 60
void Event::operator () ( ) 60
The Functor Class 60
Functor::Functor<T>(T* object, FuncType* functionPointer) 61
void Functor::operator() (Event* pEvent) 61
The Graphics System 62
IRenderer class 62
void IRenderer::Update (Screen* screen, Mouse* mouse) 63
Renderable* IRenderer::CreateRenderable ( ) 63
Renderable class 63
Renderable::Renderable( ) 64
Renderable::~Renderable( ) 64
void Renderable::OnChanged ( ) 64
Vector2D<int> Renderable::GetTranslatedPosition (int x, int y) 64
ImageResource class 64
unsigned int ImageResource::Width ( ) 65
unsigned int ImageResource::Height ( ) 65
unsigned int ImageResource::BPP ( ) 65
string ImageResource::id ( ) 65
Tying It Together 66
Appendix A – Skin DTD 70
Appendix B – Project Dependencies 72
Library Dependencies 72
Example Application Dependencies 72
Build Dependencies 72

Introduction

What is MiniUI

MiniUI is a user interface middleware library developed for C++ based application. It includes not only an XML based definition language, as seen in other user interface libraries, but also a lightweight scripting language. By combining XML for layouts and scripting for animations, a complete, portable user interface can be developed outside of the final product.

Why another UI system?

Many current user interface systems lack in the pure separation of application and user interface code. These systems embed the graphical layouts of each application hidden within the many layers of application code. This makes it difficult to maintain this interface code, or even update it for future versions of the applications.

A few examples that employ this methodology include Microsoft's Foundation Class Library, Trolltech's QT/QTopia, Linux's GTK+, and Sun's Java Swing library. All four of these libraries are extensively used in everything from rich client applications to embedded applications used in portable electronics. Although, with the right editor, all of these libraries can be used without touching the user interface code, this generated code can become hard to maintain, and very closely coupled with the application itself.

A second type of user interface libraries used today includes XML based description languages. These libraries employ some sort of parser of predefined tags to be regenerated into a user interface. Examples of this include SwiXml and SwingML for Java, K Desktop UI library for Linux, and Mozilla's XUL.

XML based user interface libraries have the benefit of separating the graphical layout from application code, which the prior examples lacked. However these XML files have the limitation of static tags only. The designer cannot create custom tags for custom widgets, at least without going back to the application code or using a complex combination of predefined tags.

To the credit of Mozilla, XUL includes a JavaScript scripting engine to overcome some of the issues of static XML. The issue, however, is JavaScript is not a lightweight scripting language and cannot be compiled for speed.

How is MiniUI different?

MiniUI takes the standard XML based user interface two steps further. It uses an XML based skin file that controls the definition of the layout file. By adjusting this file, a user interface designer has complete freedom in the tags used in the layout file. Instead of complex C++ or Java code to complete the same task, the designer can use simple rules

to define the look of a new widget. For example, if the designer wants to add in a tag named *dialogText*, the tag can be defined in the skin file. Whenever MiniUI finds this tag in the layout file, it will know to use the *dialogText* definition from the skin file to parse it.

Also, a lightweight scripting engine has been included; Lua. Lua is a great language that can be compiled into a fast and small byte code equivalent. And since Lua uses a "Just in Time" (JIT) compiler, it's still human readable. The scripting engine was included to allow for custom effects not easily defined in a basic XML format. Although the scripting engine was included for animations, it is easily used for complex child layout and XML parsing. Examples of these will be shown later.

Scripting may seem to be overly complex for what seems to be just a simple user interface library, but there's a lot of power added by such a simple library. When developing MiniUI, I found that even though the skin file allows great flexibility, it quickly became overwhelming in size due to custom layouts. By offloading that work to a script file, the skin file becomes much more manageable.

On top of the scripting language is an XPath query library that is used for parsing the layout file on load. This allows the full flexibility of complete customization of the layout file. If the widget requires custom parameters, these parameters are easily accessed through a simple XPath query.

By adding XPath into the skin file, user defined tags, which are not necessarily widgets themselves, can be parsed with ease. For example, if the current widget is a list box, as is common in user interface systems, it may want to load its content from a *content* tag. Furthermore, it may only want to load in text, so it loads from *text* tags. This can be complex to do in scripts alone, but in XPath becomes the simple: **./content/text**

Also, there is a simple animation library included that allows for a more expressive form of animation definition. Instead of calculating the current widget state every frame, a predefined animation can be loaded into the animation system. This allows for a complete hierarchy of animations to be preloaded and run automatically.

The current development process

MiniUI was built from the ground up to be expressive, expansive, and lightweight. It is a complete separation of the user interface from the application code. This allows the user interface designers to build an entire interface for an application or suite of applications without interfering with the application developers.

Since user interfaces are usually code based, traditional development requires the designers to submit lengthy requirement specifications to the developers, which in turn would build into the application code itself. This cause a major bottleneck in user interface intensive applications, such as portable electronics. This is shown in illustration 1.

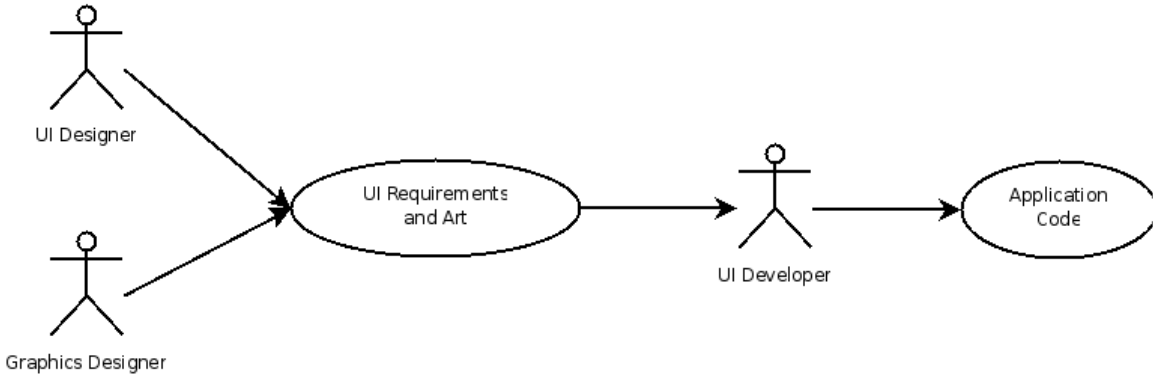


Illustration 1: Traditional Development

Instead of this slow process where the UI won't be developed for months, the designers can sidestep it and design the user interface themselves. They can then run this user interface inside a simple emulator based off of the UI library. This can be seen in illustration 2.

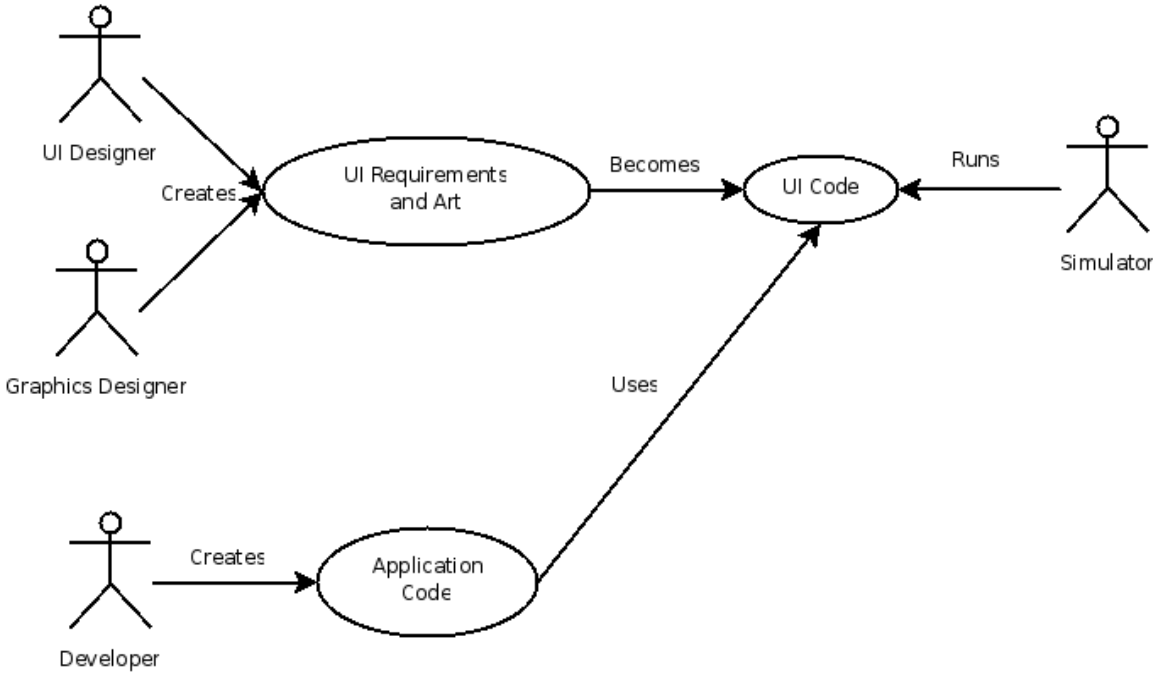


Illustration 2: MiniUI development cycle

As shown, the UI designers can have complete freedom over the interface and test them in real time well before the application developers need them. By removing the bottleneck from the development processes, more advanced user interfaces can be designed within less time.

This also helps the application developer since he or she will not have to worry about the front end interface. Since the user interface is abstracted from the application code, the developer can simply focus on what is important to him or herself, the application code.

This alone will speed up development also, since less code will have to be written by the developer.

Code based user interfaces

The baseline UI framework for most development systems is code based. This includes MFC for Microsoft Windows, Swing for Sun's Java, and QT for Linux's KDE. Because the most popular UI frameworks are code based, it's hard to see a need for another layer of separation between the UI and application code. In reality, these UI frameworks hide most of their inner workings through complex code generating tools. For example, Microsoft's Visual Studio has a graphical designer included that can easily generate thousands of lines of C++ code in the background. If the user modifies any of these lines by hand, the designer may be unable to continue parsing these files. Therefore, the designer will be unusable from then on. Improvements have been made in future versions, but it still comes down to generated code being parsed.

Any developer who has developed applications for Microsoft Windows will know about Microsoft's Foundation Classes or MFC. Applications written to MFC's standard will be nearly impossible to port to other systems since the application code written to MFC will be highly coupled to the user interface code. Direct access to widgets and their content is common. Of course, there is some separation with resource files and dialogs, but these only show the layout of the widgets, and not how they interact. The application code still needs to know that a button or text box exists, instead of an event being fired with only the information required to capture that event.

Code coupling can lessen the code reuse for future applications. In some cases, MiniUI may not be the best user interface library to use, so reduction of coupling will allow easier porting to a different user interface library. Also, reduced coupling can allow the user interface designers and the application developers to work simultaneously without interfering with each other.

Tight coupling breaks down the black box that should occur around the user interface layer. Tight coupling makes it difficult to reuse code in future applications (Gamma 24). The application should have little connection to the user interface library.

In cases where a custom integrated developing environment (IDE) needs to be used, this is unreasonable to be required. Embedded developers sometimes use custom IDEs or addons to current IDEs to fully integrate a small development environment. Sometimes these IDEs will not have code C++ based UI parsing capabilities (such as Eclipse), and the user interface will need to be designed in an external editor, or worse, by hand.

XML has the power of being extended without breaking prior versions of a parser. For example, tags can be added to the file without breaking any UI editor that currently may be using it. Therefore, if a user edits the files by hand, an IDE can still parse it properly for editing without too much issue, as long as the final XML layout is syntactically correct.

Current XML based UI frameworks

XML based UI frameworks have been popping up all over lately. They have the great benefit of being able to be only loosely coupled with the application code. Take, for example, SwiXml. It is an XML based framework for defining the Swing objects normally defined inside of a Java application. Swing normally has hard coded user interface widgets which can be difficult to update late in the development process. There are various user interface editors that can be used to define the interface, but they are still code based. SwiXML allows the designer to build the interface outside of the Java application code.

As shown, XML based tags may benefit from loose coupling to the code, but it fails when creating custom graphical elements. These XML files are strictly tied to the framework's schema. Therefore, the user interface is strictly tied to the framework's predefined widgets.

Instead of a strict schema, MiniUI doesn't use one at all for the layout XML file. Instead it uses another XML file (the skin file) to define what each tag means. The skin file contains minimal graphical layout of the widget plus any custom parameters. This way, a designer can create a new list box, and the layout file can reuse it multiple times throughout the application.

Scripting

Mozilla's XUL is currently the only popular UI framework that includes not only XML layouts, but also scripting capabilities. As this might seem like the ultimate solution for all systems, the XUL framework still has the drawback of using static, predefined tags. Some of this can be worked around using the JavaScript engine used within the framework. Although this may be good for client heavy applications, it is less than ideal for embedded systems (such as cellular phones). JavaScript is a heavy language that requires runtime parsing and complex syntax.

Instead, MiniUI uses a very lightweight scripting engine called Lua. It has many benefits over JavaScript, such as speed, just in time (JIT) compiling, size, and simplicity. It was also chosen for its tight coupling with C/C++, for easier development. The scripts can use many of MiniUI's C++ based functionality, such as XPath parsing, animations, and events.

Lua, as built into MiniUI, does have the limitation that it cannot be embedded directly in the skin or layout files. This is one thing JavaScript is good for. However, since Lua is very lightweight, this limitation is only visible initially and easy to work around.

MiniUI also uses a binding tool called luabind. Luabind creates complex links between C++ based code and the lua script. With it, the script can extend C++ based classes, create C++ objects and make method calls to them. It has a wide variety of options that MiniUI makes heavy use of. For example, the widgets used within the script are actually extensions of the actual C++ objects used within the library. That way, one unified interface can be used from within the script file and the application code (or business layer).

Luabind can be used from within the application code, but is outside the realm of this document. Documentation for luabind can be found at:
http://www.rasterbar.com/products/luabind.html.

Hardware Acceleration

MiniUI contains one aspect that none of the prior described user interfaces have. That is hardware acceleration. MiniUI makes full use out of a platform's graphics accelerator. By using the graphics processor to offload the task of drawing (many times a rather CPU intensive process), the main processor has much more time to run the MiniUI scripts and application code.

A graphics processor is a dedicated chipset to one task of drawing to a screen. It, however, is setup differently than just drawing pixels to a screen. Certain rules must be followed to draw properly. A standard user interface library not designed for a graphics processor will be impossible to make full use of one.

MiniUI, however, was designed from the ground up with a graphics processor in mind. It handles converting a widget into a series of drawing commands specific to graphics processors. This allows the user interface designer to create complete graphical user interfaces for both hardware accelerated and non hardware accelerated platforms without knowing the differences between the two.

It is the not the responsibility of the user interface designer to know the intricacies of hardware acceleration APIs, so this task was offloaded to the MiniUI library. The complete implementation of the API is hidden from both the application developer and user interface designer.

OpenGL

To utilize the graphics processor, the OpenGL (an open graphics standard defined by SGI: http://www.opengl.org/) API is used. First, the OpenGL standard is widely available in all of the most popular operating systems. It also has become the standard used in the new generation embedded graphics processors by AMD and NVidia, used in the latest cellular phones and other embedded devices.

OpenGL was also used because software reference libraries are also available for systems that don't contain hardware acceleration. This makes using OpenGL a good choice in making the user interface library cross platform, even when hardware acceleration is impossible.

The OpenGL interface is located in the /OpenGLRenderer path of the library. It contains all the code to handle MiniUI calls and convert them to OpenGL calls. MiniUI, however, is extendable. This means that it is not limited to only OpenGL and may user other hardware acceleration APIs such as DirectX for Microsoft Windows.

The OpenGL calls are abstracted from the rest of the MiniUI library. The graphics layer can be expanded to other APIs fairly easily. Since it’s separated, no changes to the application code or the MiniUI library will be needed to reconfigure the entire application for a different platform. This library extends the IRenderer interface as described later in this document. This library can actually be re-implemented to suit a completely different graphics API if needed.

SVG

SVG (Scalable Vector Graphics) was considered briefly for drawing the user interface. The issue, at least at the time of design, was that OpenVG (The SVG API), was not widely available and only a handful of graphics processors had this capability. Reference libraries were available, however, that converted these drawing commands to OpenGL. MiniUI's capabilities could be extended in the future to handle SVG style drawing.

Expansion

Of course, since MiniUI is an open platform, it allows for expanding the graphics API into other types. Even though, the example application uses OpenGL, many APIs use similar types which make them perfect for use in MiniUI. As described earlier, OpenVG can be used to replace the predrawn images with vector graphics. Also, I have come across various graphics APIs used in portable electronics (such as Trolltech's QT: http://trolltech.com/products/qt). Converting between the APIs is fairly simple and only requires overloading the draw cache, if required, and the drawing commands. This is described in the C++ API section.

Case Study: The confirmation dialog

To simplify the examples in this document, a simple case study of a confirmation dialog will be used. The dialog will be sizable based on the designer's needs, so the dialog widget itself must be dynamically sizable.

The dialog box will include two child areas. One for the text content, the other for the OK, Cancel, and/or Abort buttons. The content area should make no assumptions on what is in it, but the button area should allow for centering of the buttons and automatic spacing. This allows the designer to add only the buttons he or she needs to that dialog.

The buttons, in our examples, will be static sized to save time. They will, however have three states (No mouse hover, mouse hover, mouse down). This makes sure that the user knows when he or she has clicked or hovered over the button.

The following is the final output of a simple dialog.

As the steps involved to create a dialog divulge themselves, we'll come back to this case study to go through a complete example. It is hoped, by the end of this document, that the reader has enough information to create a complete screen to use within his or her application.

Three steps to a new UI

Currently, MiniUI is a three tiered system that uses three different types of files. It uses a skin file for changing the look and feel of the application, a layout file for changing the hierarchy of the widgets inside of a screen, and a script file for adding custom functionality to the user interface.

The skin file

Skinning is a term used when an application gives the ability to change the look and feel of an application without changing the underlying code. This was first shown to me in Nullsoft's WinAmp media player. It allowed the user to download new skin files from the internet made by other users, and load them with just a click of a button. This added a whole other realm to the application, since you could then customize the entire feel of an application. Since then, other applications took on skinning capabilities, such as Microsoft's Windows Media Player and Linux's Gnome Desktop.

The skin file, in a way, is the basic schema definition of the layout file. This XML file is parsed first to generate the parsing parameters for the layout file. In memory, an XML schema is generated for proper parsing of later XML files. A standard XML schema may have been used, but retrofitting the schema to fit the needs of MiniUI seemed overly complex. The skin file includes simpler syntax for the user interface designer, without forcing them into learning schemas.

A translation file, such as an XSLT (Extensible Stylesheet Language Transformation) could be used also to translate the layout file into a verbose definition of the user interface. Again, because XSLTs are very complex; it requires too much effort from a user interface designer to utilize such an expansive file. Plus, MiniUI has no final XML output, so there is no end file to generate from an XSLT.

MiniUI has no preconceived ideas on what widgets are or what they look like. It is up to the user interface designer to create all widget definitions in the skin file. Although this may seem like a step backwards, this design allows for ultimate freedom. Even though there are no predefined widgets, widget libraries can be included in multiple projects to use a common set of widgets.

As shown further in this document, the include tag can load prior skin files as needed. The key here is that common widget sets can be reused in multiple user interfaces. For example, if a team has multiple applications that they distribute, it only makes sense to keep the same look and feel over those applications to minimize confusion from switching between them. Of course, certain widgets may also be used in one application, so it doesn’t make sense to have that widget in the skin file for every other application. Therefore, each application can “include” the global widget set and still define their own custom widgets in their skin files.

The layout file

The layout file defines the hierarchy of the widgets. It also can contain any pertinent information needed by the skin and script files. Since the layout definition is not predefined, the user interface designer must look back at his or her skin file to determine what is correct.

This layout file also defines the screens used within the application. MiniUI applications use a series of screens, such as you would find in cellular phones or other portable electronics. These screens are what the actual application code ties itself to using the event system.

What happens, within MiniUI's library, is that the skin file is loaded and parsed into memory. As the layout file is loaded, the library goes back and references the skin file to look up tags. For example, if we defined a widget called "Button" in the skin file, the layout file can contain a tag called "Button." When MiniUI sees this tag, it finds the widget definition in the skin file and uses that as the schema for the defined tag and children.

Of course, since the skin file contains more than just a predefined widget set, the layout file may contain more than just a list of widgets. For example, if the skin file contains xpath queries to pertinent information, such as sizing and location, the layout file should contain this information. It is all dependent on the skin file and its queries.

The Script file

Since the layout and skin file are just plain XML files, the script file fills in the gaps left by the MiniUI framework. Some of these gaps may include automated child widget layout, animations and events, and any other widget specific tasks.

To alleviate "bloat" in the XML definitions, this scripting system was added. The XML files should not encompass all possible needs of a user interface. That would make the XML definition infinitely large. Instead the XML files were stripped to the bare minimum while still being functional. All other functionality was pushed to the scripting engine.

Although a C++ plugin style system could have been developed to handle this extra functionality, it would make the user interface developers and designers again dependent on the application code. Since the entire UI functionality can be developed in this scripting language, the user interface can still be run under an emulator. This means that the user interface designers don't need to learn a complex language such as C++ to move a widget or change its behavior.

Plus, it doesn't always make sense to give the user interface designer access to the original source code or compiler. When a product is released, the developers may want to allow third parties access to the user interface. This goes back to giving access to "skin" an application. As a developer of a proprietary product, you don't want to give too much access to the lower layers of the application. Of course, giving only access to the XML files gives too little access since many of the features your user interface has may be unavailable to third parties.

By giving access to the scripting language, which in itself is harmless and limited functionality besides user interface related methods, you open up the application to any number of user interfaces designed by others.

Again, opening access to an application may get a user more life out of a product. If at the end of the user's application usage lifetime, he or she finds various other user interfaces for your application, he or she might want to continue using your product over the competitions'. This also may mean more users upgrading as you continue developing the product.

Again, the skin file adds functionality that an XML only solution will assuredly lack. By opening up the user interface to custom code, many custom effects that I have not defined and possibly not even be aware are possible, can become reality.

Breaking down the skin file

The skin file is of a rather simple design. It contains a list of all the widget types and fonts. An example of a skin file is shown in figure X.

```
<skin>

  <widgets>

    <widget id="xpWindow" src="test.bmp" width="100" height="100">

      <layout>

        <section id="exampleSection">

          <source x="0" y="0" width="100" height="100" />

          <destination x="0" y="0" width="@width" height="@height" />

        </section>

      </layout>

      <children>

        <area select="./*" x="0" y="0" width="@width - 6"
height="@height - 32" />

      </children>

    </widget>

  </widgets>

</skin>
```

The Document Type Definition (DTD) file can be seen in the appendices as a reference, or to load into an XML editor to create custom skin files. As long as the skin file follows this DTD, it will be loadable into MiniUI's framework.

/skin/include

`<include file="includeFile.xml" />`

- `file: The included file`

As shown, the skin file can contain any number of included skin files. This allows the developer to include a "widget library" inside of this widget list. The included skin file takes on the same XML schema as all other skin files.

/skin/widgets/widget

```
<widget id="widgetID" src="image.png" width="100"
height="100">
     ...
</widget>
```

- **id:** The tag name used in the layout file
- **src:** Image used to draw the widget
  - May use XPath
- **width:** Width of the widget drawn
  - May use XPath
  - Optional
- **height:** Height of the widget drawn
  - May use XPath
  - Optional

The widget tag defines a widget to be used in the layout file. The id attribute becomes the tag name used in the layout file. XPath is showing for the first time here. The idea is that more expressive layout files can be used. The XPath query used in the attributes is run on the layout file at the location of the defined widget. Usually at this level, it is used to create dynamically sized widgets. For example, the layout file itself can contain the widget sizing.

Later on, the process of actually dynamically sizing the widget is shown.

The image used in the widget is not always a visual replica of the widget itself. Since a widget is pieced together from various sections, its source image can be a collection of image pieces that are rebuilt into the widget.

.../widget/layout

```
<layout>
     ...
</layout>
```

The layout section defines the pieces of the image defined in the widget tag and how they are placed within the output widget.

.../widget/layout/section

```
<section id="sectionID">
     ...
</section>
```

- **id:** Section ID for use in the script file

The section tag separates each of the sections of image that will be used to recreate the widget.

.../widget/layout/section/source

```
<source x="0" y="0" width="100" height="100">
     ...
</source>
```

- **x:** X location within the source image
  - May use XPath
- **y:** Y location within the source image
  - May use XPath
- **width:** Width of the source
  - May use XPath
- **height:** Height of the source
  - May use XPath

The source tag defines the source within the widget's image. It's a piece of the image defined by the four coordinates. Again, XPath may be used to dynamically set the source location within the image.

.../widget/layout/section/destination

```
<destination x="0" y="0" width="100" height="100">
     ...
</destination>
```

- **x:** X location within the source image
  - May use XPath
- **y:** Y location within the source image
  - May use XPath
- **width:** Width of the source
  - May use XPath
- **height:** Height of the source
  - May use XPath

The destination tag defines the destination of the source image within the final widget image. It's a piece of the widget defined by the four coordinates. Again, XPath may be used to dynamically size this source piece. The image section will be stretched or shrunk to fit inside of this destination area.

.../widget/children

```
<children>
     ...
</children>
```

The children tag contains any XPath type selection of child widgets in the layout file. Widgets may contain multi child areas, so any number of children can be added. For example, a Windows XP style window may contain a title bar, tool bar, content area, etc.

Each one of these sections can contain a number of child widgets, so should be defined as child areas.

.../widget/children/area

```
<area id="childAreaID" select="//images/*" x="0" y="0"
     width="100" height="100" />
```

- **id:** ID used in the script file for accessing children
- **select:** The query to the children in the layout file
  - Requires XPath
- **x:** X offset from the parent's location
  - Optional
  - May use XPath
- **y:** Y offset from the parent's location
  - Optional
  - May use XPath
- **width:** Width of the child area
  - Optional
  - May use XPath
- **height:** Height of the child area
  - Optional
  - May use XPath

The area tag defines the area that child widgets may be shown. The ID allows the script to lookup child areas by name to find any child widgets. The select parameter, however, is the key parameter here. It defines what in the layout file to parse for children. The power here is to define how the layout file defines its children. Any XML tags outside the realm of the child areas, in the layout file, will be ignored.

Children are parsed just like any other widget would be. The only difference is that the XPath queries of the children can access this widget's information by walking back up the XML tree. The reverse is true that the XPath queries used in this widget can be used to walk down the tree through its children.

The location parameters are there to define the actual visible area the child widgets can be shown. Child widgets will reference the offset location as its origin. If no x or y position is denoted, the child origin will be placed at the parent's origin.

The width and height attributes add another key element to the child area. It defines the drawing area that can be used. Any widgets drawn outside of this area will be cropped or removed automatically. This capability allows the children to stay within the bounds of the child area.

If the width or height attributes are not used, the children will not be cropped and therefore can be drawn anywhere. This allows the child widgets to become separated from the parent widget.

/skin/fonts

```
<fonts>
     ...
</fonts>
```

Lists all the available fonts used in the application. These fonts are made with Bitmap Font Maker by LMNOpc.com. It is a freeware bitmap font editor that can convert Windows type fonts into its own format. This format is nice since it allows for cross platform deployment without using whichever font files may be on the different systems.

.../fonts/font

```
<font id="fontID" src="fonts/MyFont.bff" />
```

- **id:** Identifier or the font
- **src:** Location of the font file

The font definition is described in the font tag. It contains the identifier and the source location of the font. The identifier is used in the script files to actually output text.

Case Study: The skin file

As was described earlier, we shall continue the case study with the definition of the skin file. As described earlier, the skin file contains the schema for the layout file. In this case study, we shall limit the focus to only the confirmation dialog, so we shall assume the basic layout of the skin file already exists (such as the font file definition). Instead, this section will focus directly on the dialog box itself without diluting the information with unnecessary information.

As was described earlier, the dialog box can be defined in any multiple of ways. It is up to the user interface designer to pick a position for the user interface and continue on with it for the entirety of the application. In our case, we shall simplify the dialog box into one simple widget. This is not an ideal way of defining a window, since it can be broken down into simpler widgets, but it shows the true power of how widgets can be defined at a large scale or small scale.

In our case, we want the layout file to contain one tag to define the dialog box. We do, however, require some parameters to build the dialog box. These parameters are listed below:

- Width: The final width of the dialog box
- Height: The height of the dialog box
- Message: Message to show in the box

To start out, we define the basic tags of the skin file.

```
<skin>

  <widgets>

    <widget>

      <layout>

        <section>

          <source/>

          <destination/>

        </section>

      </layout>

      <children>

        <area/>

      </children>
```

```
    </widget>

  </widgets>

</skin>
```

As you can clearly see, the skin file is very basic at its minimum state. What needs to be added is the widget tag in the /skin/widgets/ path. If you take a look at the widget tag described above, you'll see that it requires a size. This size parameter (width and height) are the final size of the widget in the end product.

Of course, our dialog box requires dynamic sizing. This is simple to accomplish with the use of XPath. Inside the width and height attributes, add in the XPath queries. This will search from our tag in the layout file inwards. This is shown in the following section if you wish to skip ahead.

So in the end, our final tag will look something like the following:

```
<widget id="confirmationDialog" src="casestudy/blank.png" x="@x" y="@y"
width="@width" height="@height">
```

We are not finished. Since our widget is graphical, we need to add in the layout properties of the image sections. The issue is, we can't scale the original dialog box image because it will skew the look of the icons. In our case, the question mark icon and the close dialog button should not be stretched. Instead we must be cautious on how we expand the box to fit the new content. As you can see below, if you find a sliver of an image that can stretch the content, we can use this to expand the widget to a dynamic size. As shown below, I found a sliver that stretches well with the content. This sliver will be the only part that stretches while the rest of the dialog box will stay the same. The sliver contains the same pixels going across, so the stretch should be invisible in the final out put dialog.

Of course, our example only allows expansion in the x direction. If we wanted to expand in both the x and y direction, we would have to find a center area to expand instead in a 3x3 grid of image sections.

Most of the time, a widget will have a solid area in the center that stretches well (since the solid area never changes color. Of course, there are examples where the center is not all one color, but a gradient of two colors. In this example, it makes more sense to

predefine a widget as very large with a very smooth gradient over a large area. Then MiniUI can shrink this area down smoothly using the hardware acceleration and still keep most of the gradient intact. Microsoft Office 2007 is an example of using gradients in applications.

By adding the three image sections to our original widget tag, we come up with the following.

```
<widget id="confirmationDialog" src="casestudy/blank.png" x="@x" y="@y"
width="@width" height="@height">
      <layout>
            <section id="left">
                  <source x="0" y="0" width="50" height="145" />
                  <destination x="0" y="0" width="50" height="145" />
            </section>
            <section id="center">
                  <source x="50" y="0" width="1" height="145" />
                  <destination x="50" y="0" width="@width - 301"
height="145" />
            </section>
            <section id="right">
                  <source x="51" y="0" width="251" height="145" />
                  <destination x="@width - 302 + 51" y="0" width="251"
height="145" />
            </section>
      </layout>
</widget>
```

Of course, the section tags should be explained further in this example. The source, as explained earlier, describes the section of the image used to draw the widget section. This is usually never changing due to the static nature of the original image.

The destination tag is where the true power of XPath and dynamic image sizing come into play. For example, if you look back at the broken down image of the blank dialog box, you can see that the center section is the stretchable area. In this example, the section with the id of “center” has XPath queries in the destination tag to handle dynamic sizing. As shown, the width of the output center section is sized based on the extra width not fully covered by the left and right sections.

Of course, that means that the right section needs to be moved based on the sizing of the center section. This can also be seen in the destination tag of the right section, specifically the “x” attribute.

When MiniUI loads this skin file, it uses these XPath queries to determine the final sizing and placement of the widget sections in the final output. These XPath queries can be of any nature, as long as the final output is a positive integer result.

Now we have a completed dialog box. Of course, it’s not very useful seeing as though there is no content or user interactivity. What we need to define now are child areas for sub content, such as buttons and text.

In this widget, we need three different child areas to complete the final image. One is for the buttons on the bottom right, namely the “ok” and “cancel” buttons. We also need

content for the message in the dialog box. Lastly we need some sort of text on the title bar to describe the message we are showing.

Of course, these child areas also need to move with the content area. To accomplish this, we can use XPath again to define the math. Here's an example of the new skin file content.

```
<widget id="confirmationDialog" src="casestudy/blank.png" x="@x" y="@y"
width="@width" height="@height">
      <layout>
            <section id="left">
                  <source x="0" y="0" width="50" height="145" />
                  <destination x="0" y="0" width="50" height="145" />
            </section>
            <section id="center">
                  <source x="50" y="0" width="1" height="145" />
                  <destination   x="50"   y="0"   width="@width   -   301"
height="145" />
            </section>
            <section id="right">
                  <source x="51" y="0" width="251" height="145" />
                  <destination x="@width - 302 + 51" y="0" width="251"
height="145" />
            </section>
      </layout>
      <children>
            <area id="buttons" select="//buttons/*" x="50" y="100" />
            <area id="text" select="//text/*" x="50" y="60"/>
            <area id="title" select="//title/*" x="10" y="3" />
      </children>

</widget>
```

Notice that the child area tags uses the select attribute to define which tags it allows. This, in reality, defines how the layout file is defined. The select attribute in the area tag defines the XPath query used in the layout file. These queries, like every other query used in the skin file, are relative to the widget tag in the layout file.

For example, the buttons area shows a select query of *//buttons/**. This means that under the *confirmationDialog* tag in the layout file, MiniUI will try to find the *buttons* child tag and then display any children widgets under this tag.

From the point in layout file, defined by the select attribute of the area tag, the child tags are parsed as if they were widget tags in the root path of the layout file. This is how the hierarchy of widgets is built. When child widgets are parsed, they also can have child widgets.

With the definition of the blank widget in place, the buttons and text need to be placed. To minimize the length of the example skin file, the rest of the widgets will be built in types to MiniUI. The library has text areas and image widgets built in as described later in the document.

Of course, simple images don't give of a hint that an item is "clickable." To entice the user to click one of these sections, an animation needs to be added whenever the user hovers his or her mouse or finger over the images. A simple animation, such as

highlighting the button, will be added. To simplify this task, a new widget needs to be defined.

```
<widget id="HoverHighlight" width="/HoverHighlight/*[1]/@width"
height="/HoverHighlight/*[1]/@height">
      <children>
            <area id="area" select="*" />
      </children>
</widget>
```

This widget is simple in design, but it has a complex xpath query defining the width and height. What's happening here is the width and height are read from its first child widget. This simplifies to highlighting the area of the first child. This will be shown later in the layout case study.

Breaking down the layout file

As described before, the layout file contains the hierarchy of widgets and screens used in the user application. It also does not have any inclination on its own schema, as the schema is dynamically generated by the application's skin file(s).

There are, however, some standard tags that must be used to define the screens. The screens are seen similar to a window in a windowing operating system (Windows, OSX, etc.). Although making a windowed environment may seem the proper methodology to use, some environments don't allow for windows. Games and some embedded systems dynamically generate all the content on the screen without the help of a windowing environment. Therefore, the design was to simplify to a screen level so both environments can use them. The windowing environment can add a simple window wrapper around the screen. This can be seen in the example application.

This also helps in cross platform development. Since MiniUI does not define a way of creating a window, it makes no assumption on the requirements of that window. The developer can design any window wrapper that fits with their current windowing environment without much work on the MiniUI side.

/interface/screen

```
<screen id="MyScreen">
     ...
</screen>
```

- **id:** The identifier for that screen.

The screen tag defines each of the screens used in the application. The identifier is used in the C++ code to handle events for that screen. For example, if the screen is our dialog box, the C++ code behind needs to know that the events are being fired for its screen(s).

The children tags of this XML tag are the widgets defined by the skin file. The parsing of these tags is defined in the skin file section of this document.

Case Study: The layout file

Again, we come back to the case study of the dialog box. The layout file is much simpler compared to the skin file due to the fact that the skin file contains most of the definition. In this case, the skin file should be referenced directly, since it contains the definition of the layout file.

```
<widget id="confirmationDialog" src="casestudy/blank.png" x="@x" y="@y"
width="@width" height="@height">
      <layout>
            <section id="left">
                  <source x="0" y="0" width="50" height="145" />
                  <destination x="0" y="0" width="50" height="145" />
            </section>
            <section id="center">
                  <source x="50" y="0" width="1" height="145" />
                  <destination   x="50"   y="0"   width="@width   -   301"
height="145" />
            </section>
            <section id="right">
                  <source x="51" y="0" width="251" height="145" />
                  <destination x="@width - 302 + 51" y="0" width="251"
height="145" />
            </section>
      </layout>
      <children>
            <area id="buttons" select="//buttons/*" x="50" y="100" />
            <area id="text" select="//text/*" x="50" y="60"/>
            <area id="title" select="//title/*" x="10" y="3" />
      </children>

</widget>
<widget id="HoverHighlight" width="/HoverHighlight/*[1]/@width"
height="/HoverHighlight/*[1]/@height">
      <children>
            <area id="area" select="*" />
      </children>
</widget>
```

This skin file defines two main widgets, the confirmation dialog, and the hover highlight area. We'll start by defining the confirmation dialog.

```
<confirmationDialog id="Dialog" x="10" y="10" width="302">
</confirmationDialog>
```

The tag name used is the same as the id of the widget in the skin file. This allows the layout DTD to be customized to the skin file. The width attribute was added since the skin file widget tag defines an XPath query that requires it.

Adding in the child widgets, as defined by the child areas selectors, creates the following final layout:

```
<confirmationDialog id="Dialog" x="10" y="10" width="302">
      <title>
            <TextArea id="title" font="TitleFont" text="Confirmation
Dialog" />
      </title>
```

```
      <text>
            <TextArea id="dialogtext" font="DialogFont" text="Please
confirm this dialog" />
      </text>
      <buttons>
            <HoverHighlight outOpacity="0.5" overOpacity="1.0" x="0"
y="0">
                  <Image id="okayButton" src="casestudy/Cancel.png"
width="84" height="36" />
            </HoverHighlight>
            <HoverHighlight outOpacity="0.5" overOpacity="1.0" x="100"
y="0">
                  <Image id="cancelButton" src="casestudy/okay.png"
x="0" y="0" width="89" height="36" />
            </HoverHighlight>
      </buttons>
</confirmationDialog>
```

Now it's visible where the XPath queries are used. The button area has the contents of the two images, surrounded by the HoverHighlight widget. The text area and title also have their own child areas.

The TextArea tag and Image tag are predefined in MiniUI. They can be used without defining them in the skin file. You may have also noticed that the hover highlight area has extra attributes that were not defined in the skin file, namely the out and over opacity. This is okay since the XML parser skips over any attributes it doesn't directly recognize. In this case, however, the script file requires extra parameters. The script files can also load parameters directly from the layout file, like the skin file does.

Breaking down the script file

The script file is where all the magic occurs of running the back-end systems in the user interface that does not belong to the business logic of the application. Since the scripts contain the most complexity, this section will be, by far, the largest within the document. Scripts handle everything from simple events and animations to complex layouts and layout file parsing.

As described earlier, MiniUI uses Lua as its scripting language of choice. It is beyond the scope of this document to define the Lua language, so it is assumed the reader has a basic understanding of object oriented languages and Lua. Detailed documentation for the Lua language can be found at http://www.lua.org/manual/5.1/.

It is expected that the user interface designer will use Lua's *require* function to include any other script files in the main script. There is no other method for loading in external script files.

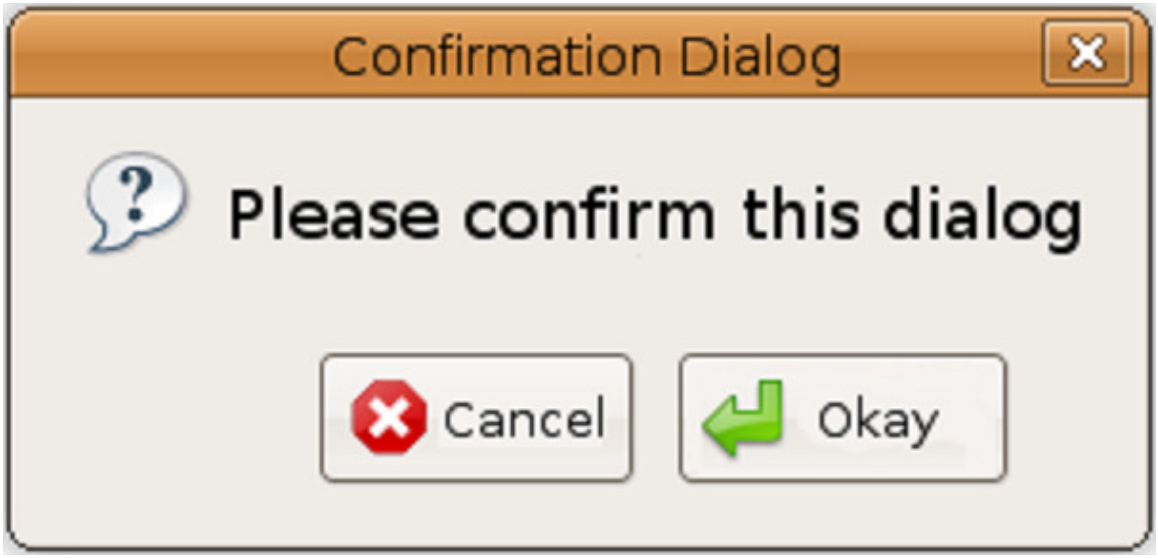
Screen space versus widget space

In this document, references will be made to screen space and widget space. This may be a confusing at first. Screen space is the defined space of the screen where (0,0) is the top left corner. This is handled by the operating system in a windowing environment, or is static in most embedded systems.

Widget space is handled much differently. Each widget has its own widget space defined by its position, size, scale, and rotation. As shown in figure X, each widget space conforms to the location, rotation, and scaling of the widget itself.

In this screenshot, the cancel button is located at location 82, 92 in pixels.

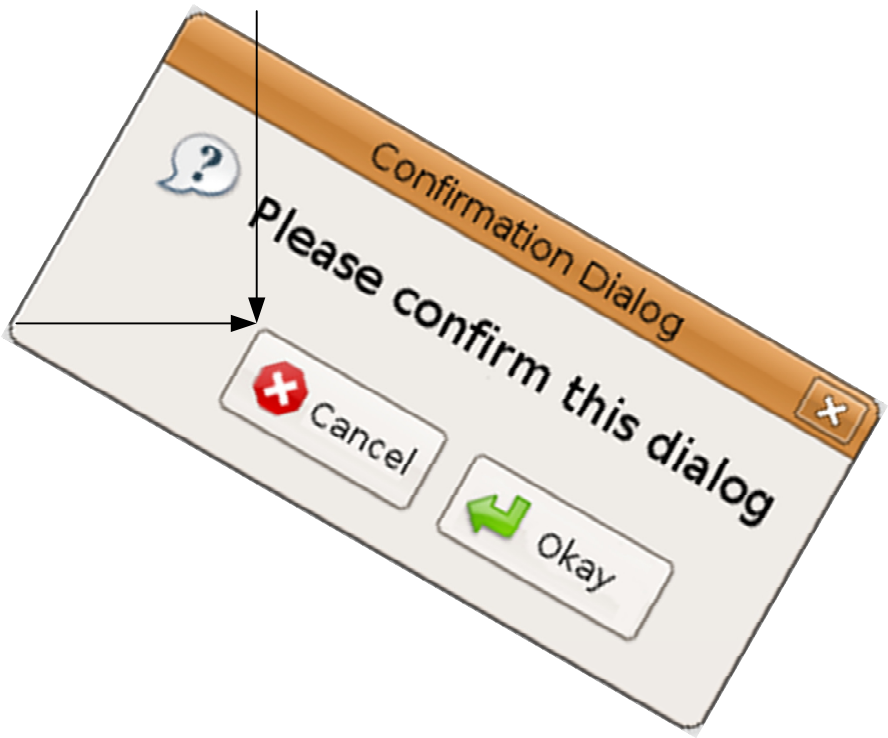
For example, in a normal case, if a mouse moved one pixel down, the coordinate would add one to the y location. If, however, the widget was scaled to two times the original size as shown in figure X, the mouse would have to move two screen pixels to add one to the y location, like before.



In this example, the location of the cancel button moves to location 164, 184.

This reasoning becomes apparent quickly in a rotation example. Take a look at figure X. It shows two dialog boxes with OK buttons. In the first case, the button is offset X pixels down, and X pixels over. The parent widget knows this by checking the widget location. If the dialog was rotated thirty degrees to the right, the new location would be at 120 pixels down and 117 pixels to the right from the origin. This is defined by the equations:

$$x = \cos\theta * x_{original} + \sin\theta * y_{original}$$
$$y = \cos\theta * y_{original} + \sin\theta * x_{original}$$



As you can see, this equation is not just a simple algebraic equation, but requires the use of sines and cosines. This can become complex to handle in extreme cases such as scaling and rotating. Instead, MiniUI handles all the conversion of the mouse coordinates to widget space in the backend. Therefore, when the mouse hovers over the screen space location of the drawn widget, MiniUI will convert this back to widget space so the widget itself knows the mouse is hovering over the cancel button widget located at (82, 92) where the parent originally placed it.

Child widgets inherit the widget space of the parent's child areas. Therefore, if the parent widget is scaled to two times its original size, the children will also be scaled by that factor. The benefit here is that a parent widget can be moved, scaled, or rotated and all the children will always stay with it as one entity.

Widget space is a similar concept to object space in OpenGL. In fact, the concept was ported over from the OpenGL specification. Each separate entity on screen can have its own local coordinate system so all rotations, scaling, and translations are dependent on the local coordinate system of that on screen object (OpenGL Programming Guide).

This duplication of ideas was put in place to help to ease usage of hardware acceleration. The widget space can be translated directly into OpenGL object space used by hardware accelerators. This allows full usage of the hardware acceleration in the user interface.

The widget class

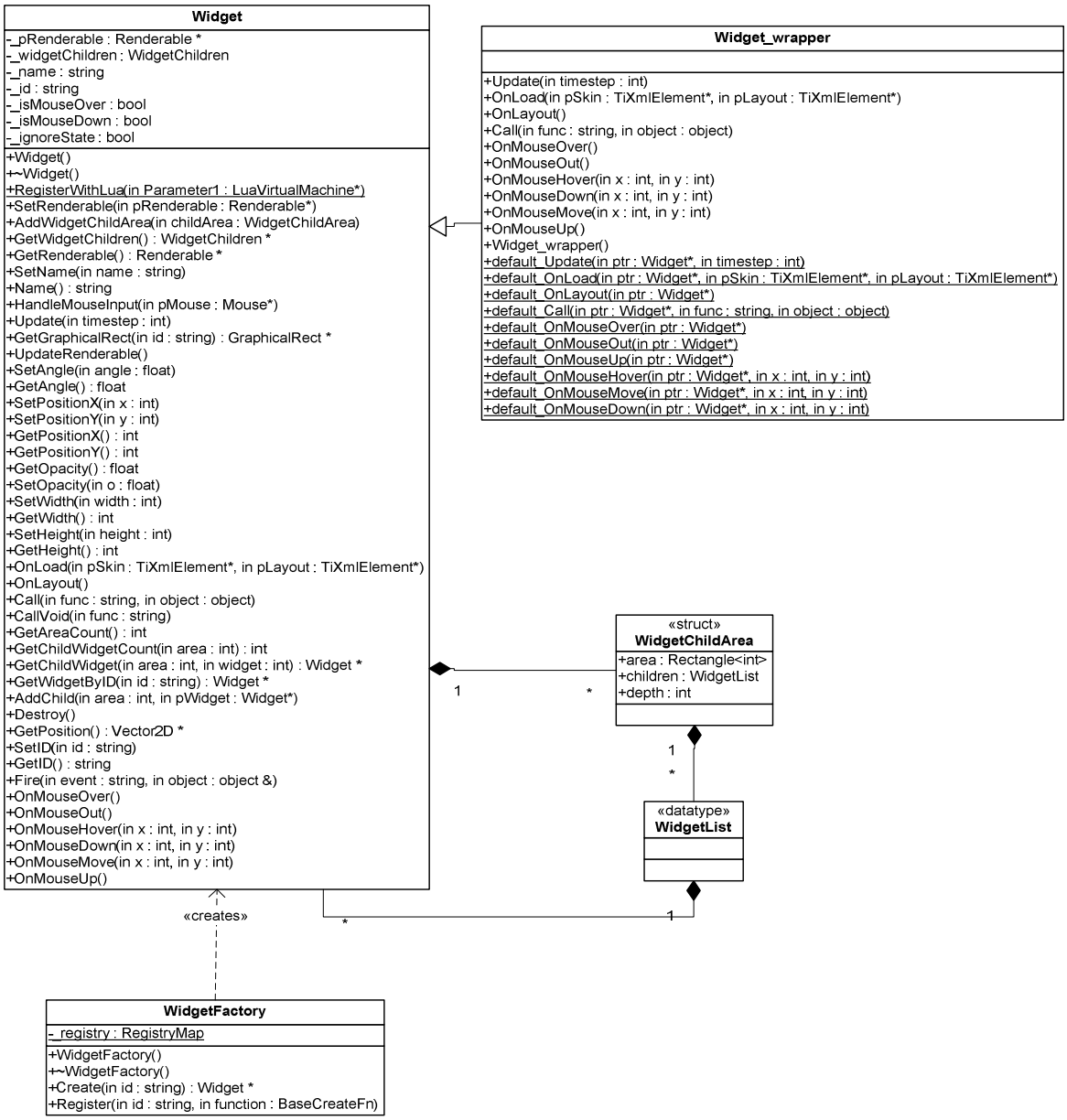
To start out, the widget class is the base class of fall the objects used by MiniUI. When MiniUI creates a widget based off of the layout and skin files, it searches the scripts for a class that resembles the widget. If the MiniUI library finds a class the matches the widget's definition, it instantiates that class. What this means to the designer is that he or she can override the default functionality of the widget. If MiniUI, however, does not find a user defined class, it uses the base widget class instead.

In the backend, the widget class is a C++ based class. It contains the basic functionality of the widget, such as position, angle, transparency, etc. Certain methods and attributes are exported to Lua for overriding or use. The Lua object is a polymorphic object, so not all methods need to be overridden, only the required methods.

As shown in the UML diagram below, a wrapper class "Widget_wrapper" was put into place to over load methods in Lua. Luabind's requirements forces overloaded classes to include a wrapper class to call member methods. When overloading a widget class in Lua, in actuality, the class' base class is Widget_wrapper. This is done because in C++ there is no simple way of having a function pointer to an object's method.

To define a widget subclass in Lua, use the *class* function. It references the *Widget* class as its base class, and will inherit all the exported base methods and attributes.

Once this is done, when a widget by the name of *MyWidget* is found in the user interface, it will create an instance of *MyWidget* instead of *Widget.*



.

Widget:__init()

The constructor method must be overridden in subclasses, and must call the *super()* method to construct the superclass. Minor other initialization code can be placed within the constructor method, but note that the XML files have not been parsed completely at this point.

Widget:OnLoad(skinElement, layoutElement)

- **skinElement:** XML node to the widget in the skin file
- **layoutElement:** XML node to the widget in the layout file

This method is used to parse the XML files in the script. There are two XML nodes passed in, the skin (which is the node in the skin file containing this widget), and the layout (which is the node in the layout file currently being parsed that pertains to this instance of the widget). Override this method to do custom parsing.

At this point, the XML files are parsed far enough for MiniUI to get the basic information, however no children widgets have been created. Therefore it is normal to put the current widget's initialization code, but ignore the child widgets.

Any information that the child widgets may need should be initialized here also. After the *OnLoad()* method is called, the children are created.

XML information can be pulled out from the skin and layout file from this point. Basically, if a custom parameter is needed for a widget inside the layout file, use the XPath component to retrieve it from the layout file.

Widget:OnLayout()

Now that the XML files have been parsed and the child widgets created, the current widget can now layout the children in any manner it wants. This is the time to iterate all the child widgets and place them so they do not overlap. A widget defaults to position (0, 0), so all the children will be placed on top of one another if not moved.

Widget:OnMouseOver()

This method is called when the mouse has just moved over the widget since the last frame.

Widget:OnMouseOut()

This method is called when the mouse has just moved out from within the widget since the last frame.

Widget:OnMouseHover(x, y)

- **x:** X location relative to widget space
- **y:** Y location relative to widget space

This method is called every frame while the mouse is within the dimensions of the widget. The x and y parameters are relative to the widget space. In other words, the top

left corner of the widget is always (0, 0). Also, if the widget is rotated at any degree, the coordinate system will also be rotated to the same degree. This allows the UI designer to handle cases where the widget may actually be moving or rotating freely while the user handles it.

Widget:OnMouseMove(x, y)

- **x:** X location relative to widget space
- **y:** Y location relative to widget space

This method works similar to *OnMouseHover()*, except it occurs only when the primary selector (usually the left mouse button) is activated. This is useful when handling drag and drop cases.

If the mouse moves outside of the widget space while the primary selector is activated, the *OnMouseMove()* is still run until the user deactivates the selector. So it is not uncommon to see negative positions. It is the job of the script to handle those properly at any time.

Widget:OnMouseDown(x, y)

- **x:** X location relative to widget space
- **y:** Y location relative to widget space

This method is called when the primary selector has been activated within the widget space. It is only called once per mouse down event.

Widget:OnMouseUp(x, y)

- **x:** X location relative to widget space
- **y:** Y location relative to widget space

This method is called when the primary selector has been deactivated subsequently after an *OnMouseDown()* event. It does not matter if the mouse has moved outside of the widget space, it will be called after every *OnMouseDown()* event that occurred within the widget.

Widget:Call(function, object)

- **function:** Function to call from a lookup table
- **object:** An encapsulated object containing the function parameters

Binding script methods to C++ code can be complex. To simplify this functionality, the *Call()* method was introduced. When calling a method in the script (from C++), this method will be called. *function* will be the name of the function to call while *object* will contain all the parameters to that function. It is up to the user interface developer to use a

lookup table to determine which method to call based on the *function* and/or the object parameters.

Widget:Update(timestep)

- **timestep:** Time, in milliseconds, since last call. Zero on first call.

MiniUI focuses on smooth, continuous animations. To achieve this effect, things have to be done every frame. To achieve this, the user interface developer can overload the Update method, which is called every frame. Here it can run its own custom animation code, or manually update the animation class object described later.

Timestep is the time, in milliseconds, since the last frame occurred. Usually the developer wants to handle animations in a framerate independent way. The developer can calculate the next animation frame based on the timestep value. This is the same way the *Animator* class handles animations.

Update is only called when the widget's screen is currently being shown. If it is hidden by another screen, update is never called. It is also worth noting that the first time *Update* is called, timestep will be zero.

Widget:Fire(event, object)

- **event:** C++ style event to be fired in the application space
- **object:** Encapsulated object containing the event parameters

Running fancy animations isn't very useful if that's all the user interface does. Eventually, the user interface will want to notify the business layer that some event has occurred. To handle this, the *Fire* method can be called with the event name and any object containing the event parameters.

An event will fire in the C++ code at the screen level. This is described more in the event system section of this document.

Widget:GetWidgetByID(id)

- id: The ID of the widget to search for

In the script, the user interface developer may need to quickly find a child widget in one of the many levels of hierarchy of widgets. This method will search starting at the current hierarchy level and drill down until it finds the widget that has the indicated ID or it exhausts its search. If no widget containing that ID is found, a *nil* value is returned.

Widget:GetChildWidget(area, widget)

- **area:** Which area to search

- **widget:** Index of the child widget in the child area defined.

To gain access to a child widget, the script can call *GetChildWidget()*. Remember from the skin section that a widget can have multiple child areas. An *area* value must be sent it to denote which child area to look into. This can be either the index value of the area based on the order they were defined in the skin, or it can be the id of the area.

Widget:GetChildWidgetCount(area)

- **area:** The child area to get the child count.

Many times, the designer may not know how many children a child area will contain when developing the the script files. To counteract this issue, the Widget class has a method for finding out the number of children one level down in the child area.

Again, area can either be an index value of the area based on the order in the skin file, or it can be the ID of the area.

```
local area = 0;
local childCount = self:GetChildWidgetCount (area);
```

Widget:GetGraphicalRect(id)

- **id:** Identifier of the rectangle to access.

As shown in the skin file section, a widget is split into graphical sections that are pieced together at runtime. To access these graphical "rectangles," the script can grab the widgets graphical section by its ID.

The *GraphicalRect* class is described later in this document.

Widget:UpdateRenderable()

Since MiniUI is a hardware accelerated scripting engine, many pieces of the user interface are actually stored inside the computer's or embedded device's graphics accelerator chipset. As noted earlier, this helps decrease the latency of drawing which increases the speed of the entire system.

Sometimes, a widget's changes are drastic enough to force the user interface to reload that widget into the graphics processor. An example of this change is when updating anything at the GraphicalRect level. Since a widget's drawing commands are cached, any change to these will require the drawing commands to be updated in cache.

In non-accelerated environments, this command does nothing. However, since the user interface is cross platform, it is always a good habit to run this command after making any drawing command changes.

Any movement, scaling, rotating, or changing of transparency does not require running *UpdateRenderable()*. These drawing commands are not cached to the graphics processor so they can be frequently updated by the scripts.

Widget:AddChild(area, widget)

- **area:** The child area of the parent widget to add to.
- **widget:** The widget to add

Sometimes, widgets need to be dynamically created. This method allows the developer to add a child widget to the current parent. The area is the id or index of the child area, where the widget is a newly created widget object.

Widget:angle

This property sets the angle of the widget relative to its parents widget space. The angle is set in degrees from vertical, rotating around clockwise.

Widget:x and Widget:y

The x and y parameters are the positioning values. When set, the widget will be placed in the x and y coordinates relative to its parent's child area. Similarly, if read, returns the coordinates relative to the parent's child area.

Widget:widget and Widget:height

These parameters get or set the widget's dimensions. Setting these values does *not*, however, change the physical dimensions of drawing. It is up to the script to properly set the graphical rectangles to the new proper dimensions. These parameters are usually only used for widget layout in a parent.

Widget:opacity

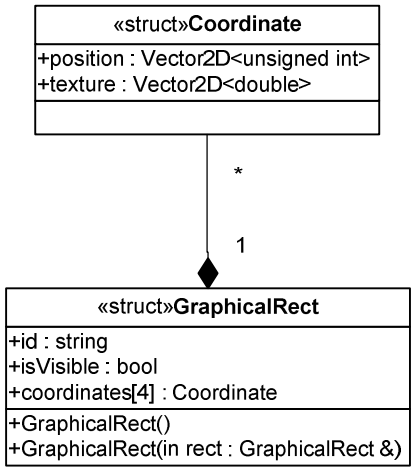
The opacity parameter sets the entire widget's transparency level. When set to a value of 1.0, it is completely opaque. When set to 0.0, it becomes completely transparent. Setting a value in between these will cause the widget to become semi-transparent.

Note that making a widget semi-transparent does not slow down the user interface. Since hardware acceleration handles transparency well, it can be used at any time without side effects.

Widget:id

This parameter gets or sets the widget's ID. Although it is possible to set the ID dynamically at run time, it is advised not to use this feature.

GraphicalRect class



A graphical rectangle is the lowest level drawing component used in MiniUI. A widget may contain one or more of these components to draw the entire widget.

These objects are cached in the graphics accelerator card, so it should be understood that any changes to these are considered an expensive operation. This is because when a value is changed and the widget is updated, the entire widget is re-cached. The graphics accelerator chip must be interrupted and sent new drawing commands.

Although modifying a graphical rectangle is considered an expensive operation, real-time use of this mechanism is still possible without noticeable speed decreases. It must be understood that the graphics processor could handle dozens of individual widgets real-time modification. Anything above that, which of course is not a concrete number, may cause noticeable delays in drawing.

GraphicalRect:id

This property gets the identifier of the graphical rectangle.

GraphicalRect:isVisible

This property gets or sets the visibility of the graphical rectangle. This allows a single graphical rectangle to be hidden at any time.

Although this may not seem useful, an entire animation's worth of frames can be placed into an image, loaded into a widget, and the script could animate between frames only by setting the visibility of each individual graphical rectangle or frame.

MiniUI class

LuaSystem::**Helper**
+CreateWidget(in widget : string) : Widget * +RegisterWithLua(in pVM : LuaVirtualMachine*)

The MiniUI class is a simple helper class for added functionality.

MiniUI:CreateWidget (widgetID) [static]

- **widgetID:** The identifier of a widget

This static method allows the script developers to create widgets dynamically that are accessible through C++ and Lua. Since creating a widget through the *new* function in Lua will only generate a script object (and not the corresponding C++ object), this helper function is needed.

xpath class

One of MiniUI's strongest assets is its ability to access XML and use XPath. The scripting files were also given this ability to further enhance the widget's pluggable parameters. During a widget's creation, the *OnLoad()* method is called with two parameters. These two parameters contain the XML elements in the skin and layout file.

Instead of digging in deep into the many layers of the XML using functions such as *GetChild()*, XPath was included. XPath allows the developer to search for nodes or attributes at a specific location in the XML file.

xpath:ToElement(xmlElement, path) [static]

- **xmlElement:** The XML node to search from.
- **path:** The XPath search string

This method allows the developer to find an XML node based on the XPath query string. It returns the node found, or *nil* if not found.

xpath:ToString(xmlElement, path) [static]

- **xmlElement:** The XML node to search from.

- **path:** The XPath search string

This method returns a string representation of the xml item based on the xpath search query. If an element is chosen, its tag name is returned. Otherwise its the text value at the xpath position.

xpath:ToNumber(xmlElement, path) [static]

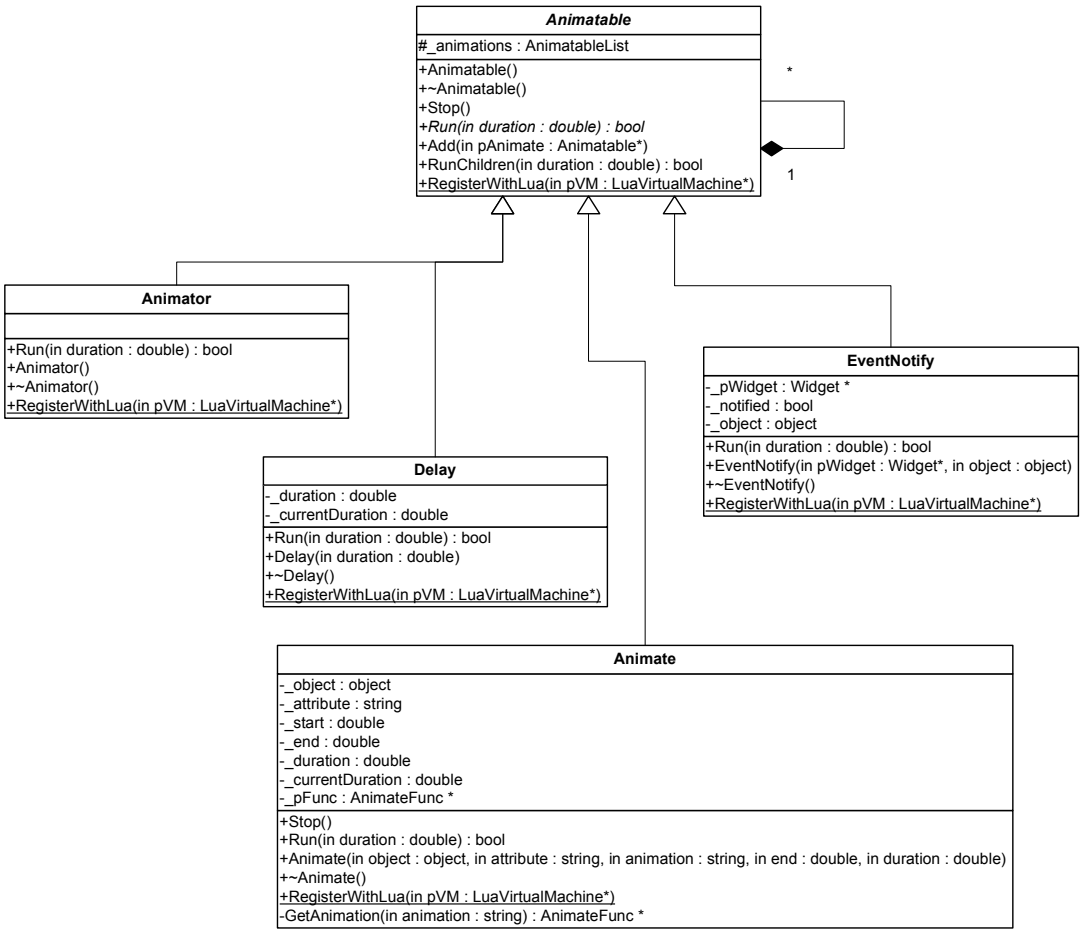
- **xmlElement:** The XML node to search from.
- **path:** The XPath search string

This is the same method as above, but it converts the string to a numeric (double precision floating point) value.

Breaking down animations

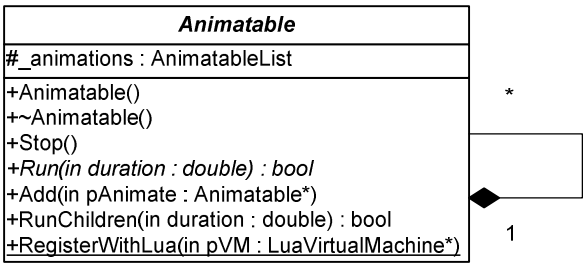
Although animations are technically a part of the scripting engine, they are so important to the MiniUI system that they deemed their own section in this document.

MiniUI's animation system was built from the ground up to be simple, intuitive, and fast. There is no point to develop a custom coded animation system for each type of widget. Widget's themselves have very similar attributes that can be animated, or a widget's custom parameters can be modified.

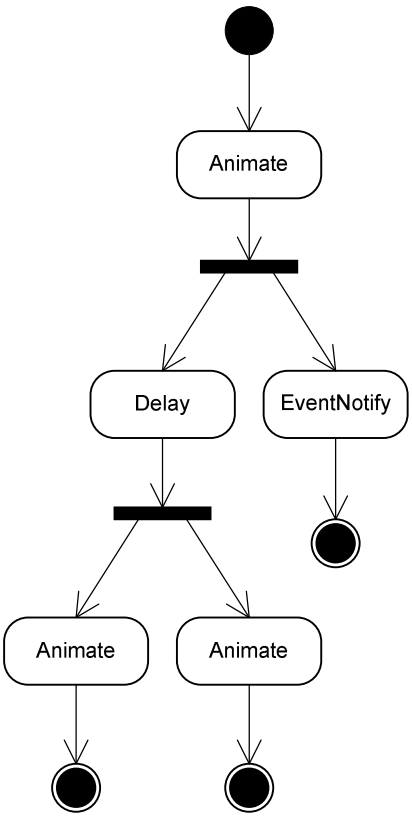


The MiniUI animation system is defined as a main animation object, the *Animator*. It contains multiple animations, *Animatable*s, in a tree like structure. Animatables can include actual animations of an attribute, delays that wait for a specific time limit, or event notifications to notify the widget that the animation is in a specific state.

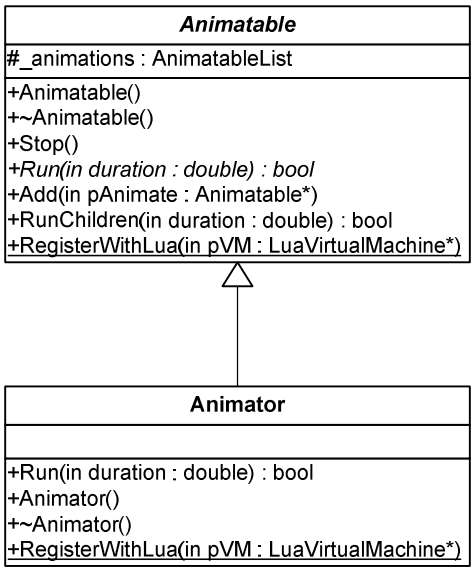
By looking at the UML class diagram of the animation system, it is clearly shown that an *Animatable*, can contain zero or more children animations. These animations will be called concurrently only after the current *Animatable* object is finished. The important aspect of this is animations can be chained together to become a more complex animation.



Plus two or more parameters can be modified concurrently. Likewise, the same parameter can be modified in more than one way. Any combination of these defined animation sequences can cause complex animations to occur with little effort by the developer or user interface designer.



Animator class



The *Animator* class defines the main, public functionality of the animation system. This class is the one used by the script to attach new *Animatable* objects to the animation system and run the current *Animatable* objects.

Animator( ) [static]

The constructor creates a new *Animator* object. Each widget that uses the animation system must create its own *Animator* object.

Animator:Run(duration)

- **duration: Time since last time Run was called**

To run the animation for the current frame, the script must call *Run* every frame. This allows the animation system to handle anything it needs to and returns back. The developer does not need to know if any animations still exist, or which animation the *Animator* is currently on. The *Run* method will handle all of this seamlessly.

Since the run method needs a duration, it is best to run this method during the widget's *Update* method. The duration passed into the widget's *Update* method can be passed directly into the *Run* method.

Animator:Stop()

Sometimes during the life of an animation, the animation must be stopped and cleared. This method will stop the current animation at its current point, and clear out any child animations it may still have running.

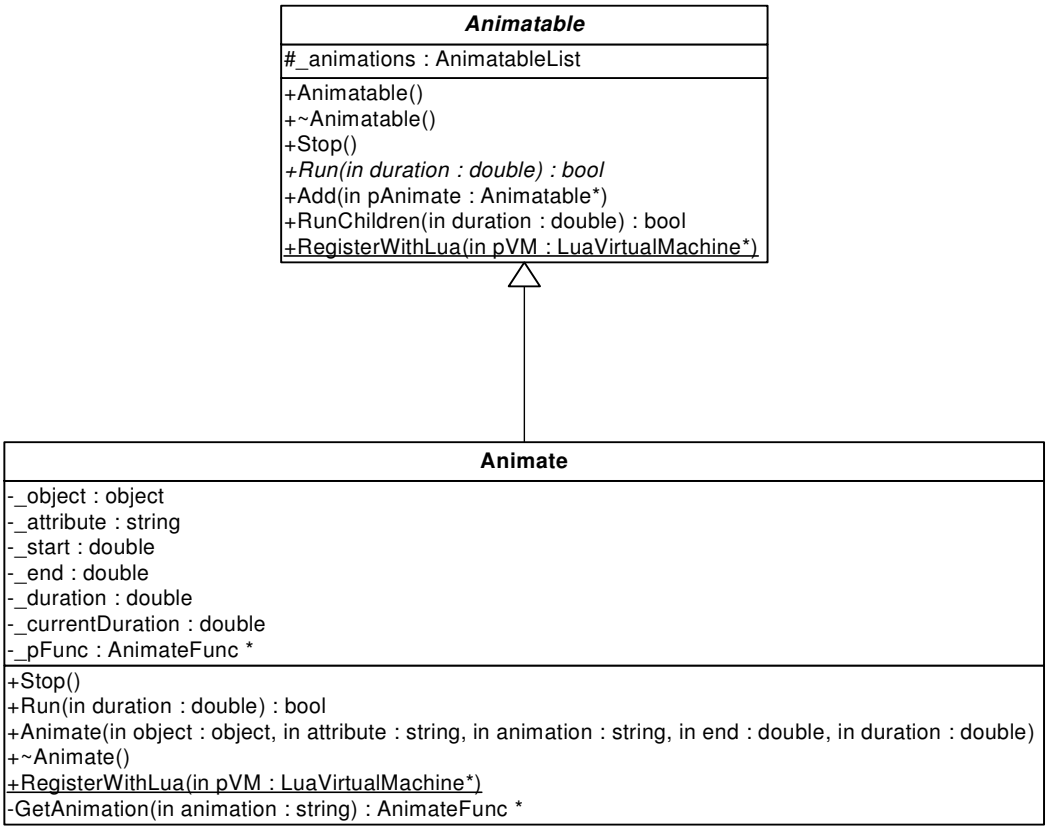
It is safe to run *Stop* at any time if the animation system has *Animatables* or not.

Animator:Add(animatable)

- **animatable: Animation object to add to the current animation parent**

Adding an animation to the *Animator* is simple by running the *Add* method. It will take the *Animatable* and add it to the currently running animations. The next time *Run* is called, it was run this *Animatable*.

Animate class



The *Animate* class is simplest form of animations. It defines changing one attribute from its current state to a different state through a series of *Run* commands. The changing attribute will be transitioned smoothly through a predefined set of transition types. These include, but are not limited to:

- Linear transformation – Changing of the attribute in a linear fashion
- Sine in and out transformation – Changing of the attribute with a speed up at the beginning of the animation, and a slow down at the end to smooth out the transitions.
- Bounce In – Bounces off the starting point in a spring like fashion
- Bounce Out – Bounces off the ending point in a spring like fashion.

Animate(object, attribute, animation, end, duration) [static]

- **object** – Object to animate
- **attribute** – Attribute of the object to animate.

- **animation** – Animation name
- **end** – End point of attribute
- **duration** – How long, in milliseconds to run the animation

The constructor creates a new *Animate* object. It is the life of the animation system. It defines each individual movement in a complex sequence of animations. The *animation* attribute can be one of the following predefined names:

- linear – Linear transformation
- sineInOut – Sine in and out transformation
- bounceIn – Bounce in
- bounceOut – Bounce out.

The linearity of the movement will be based off of the animation attribute. The animation always starts from the current attribute's value. This is why the start point is never defined. The *Animate* object will interpolate between the attribute's current value and the end point over the duration of time defined.

Animate:Run(duration)

- **duration: Time since last time Run was called**

To run the animation for the current frame, the script must call *Run* on the parent *Animator* class every frame. This allows the animation system to handle anything it needs to and returns back. The developer does not need to know if any animations still exist, or which animation the *Animator* is currently on. The *Run* method will handle all of this seamlessly.

The *Animator* class will in turn run this method using the duration calculations. The duration is always started from zero no matter how far into the animation tree the current *Animate* object lies.

Since the run method needs a duration, it is best to run this method during the widget's *Update* method. The duration passed into the widget's *Update* method can be passed directly into the *Run* method.

Animate:Stop()

Sometimes during the life of an animation, the animation must be stopped and cleared. This method will stop the current animation at its current point, and clear out any child animations it may still have running.

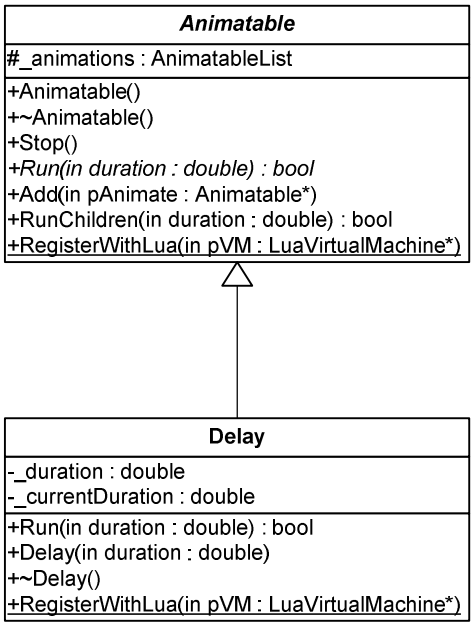
It is safe to run *Stop* at any time if the animation system has *Animatables* or not.

Animate:Add(animatable)

- **animatable: Animation object to add to the current animation parent**

Adding an animation to the *Animate* object is simple by running the *Add* method. It will take the *Animatable* and add it to the currently running animations. The next time *Run* is called, it was run this *Animatable*.

Delay class



The *Delay* class is the predefined way of placing a pause in an animation. It's also used to synchronize the timings between animations if needed. Mostly, it delays execution of the children animations until the duration time is passed.

Delay (duration) [static]

- **duration** – Time to wait

This constructor simply creates the delay object to be added to an animation sequence. Since it's based off of the *Animatable* class, it can be added to the sequence just like an *Animate* object.

Delay:Run(duration)

- **duration: Time since last time Run was called**

To run the animation for the current frame, the script must call *Run* on the parent *Animator* class every frame. This allows the animation system to handle anything it needs to and returns back. The developer does not need to know if any animations still exist, or which animation the *Animator* is currently on. The *Run* method will handle all of this seamlessly.

The *Animator* class will in turn run this method using the duration calculations. The duration is always started from zero no matter how far into the animation tree the current *Animate* object lies.

Since the run method needs a duration, it is best to run this method during the widget's *Update* method. The duration passed into the widget's *Update* method can be passed directly into the *Run* method.

Delay:Stop()

Sometimes during the life of an animation, the animation must be stopped and cleared. This method will stop the current animation at its current point, and clear out any child animations it may still have running.

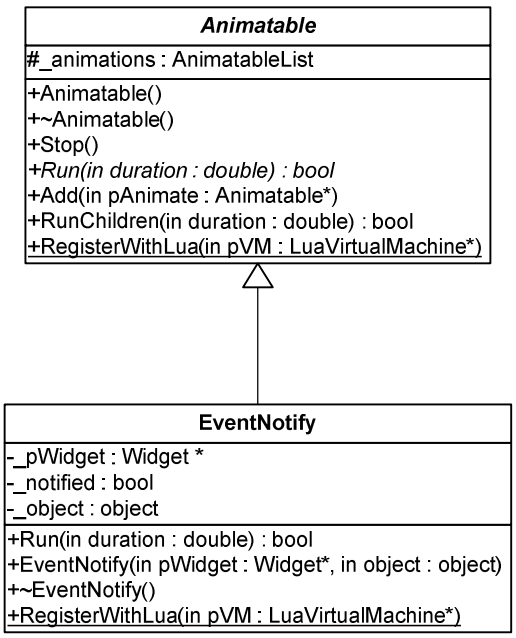
It is safe to run *Stop* at any time if the animation system has *Animatables* or not.

Delay:Add(animatable)

- **animatable: Animation object to add to the current animation parent**

Adding an animation to the *Delay* object is simple by running the *Add* method. It will take the *Animatable* and add it to the currently running animations. The next time *Run* is called, it was run this *Animatable*.

EventNotify class



The *EventNotify* class adds a callback mechanism during an animation. If the user interface developer wants notification that an animation sequence is done (or is at a specific state), an *EventNotify* object can be added to make a *Widget:Call* method call.

EventNotify animations take zero duration, so it is safe to place them within an animation sequence while still keeping in sync with other currently running animations.

EventNotify (object, event) [static]

- **object** – Object to notify
- **event** – Event to send to object

This constructor simply creates the *EventNotify* object to be added to an animation sequence. Since it's based off of the *Animatable* class, it can be added to the sequence just like an *Animate* object.

EventNotify:Run(duration)

- **duration: Time since last time Run was called**

To run the animation for the current frame, the script must call *Run* on the parent *Animator* class every frame. This allows the animation system to handle anything it needs to and returns back. The developer does not need to know if any animations still exist, or which animation the *Animator* is currently on. The *Run* method will handle all of this seamlessly.

The *Animator* class will in turn run this method using the duration calculations. The duration is always started from zero no matter how far into the animation tree the current *Animate* object lies.

Since the run method needs a duration, it is best to run this method during the widget's *Update* method. The duration passed into the widget's *Update* method can be passed directly into the *Run* method.

EventNotify:Stop()

Sometimes during the life of an animation, the animation must be stopped and cleared. This method will stop the current animation at its current point, and clear out any child animations it may still have running.

It is safe to run *Stop* at any time if the animation system has *Animatables* or not.

EventNotify:Add(animatable)

- **animatable: Animation object to add to the current animation parent**

Adding an animation to the *Delay* object is simple by running the *Add* method. It will take the *Animatable* and add it to the currently running animations. The next time *Run* is called, it was run this *Animatable*.

MiniUI class

The MiniUI class is a simple class for special global functionality without modifying the global script space. These helper functions are added to help the script do some basic tasks.

MiniUI:FindWidget(id) [static]

This method simply finds the first widget with the ID of *id*. It searches in the current screen and drills down the hierarchy until it finds the widget, or exhausts its search.

Case Study: The script file

To complete this case study, the script needs to be added to control the animations. In this case, only the hover highlighter widget requires any scripting. Again, we look back at the layout file.

```
<confirmationDialog id="Dialog" x="10" y="10" width="302">
      <title>
            <TextArea id="title" font="TitleFont" text="Confirmation
Dialog" />
      </title>
      <text>
            <TextArea id="dialogtext" font="DialogFont" text="Please
confirm this dialog" />
      </text>
      <buttons>
            <HoverHighlight outOpacity="0.5" overOpacity="1.0" x="0"
y="0">
                  <Image id="okayButton" src="casestudy/Cancel.png"
width="84" height="36" />
            </HoverHighlight>
            <HoverHighlight outOpacity="0.5" overOpacity="1.0" x="100"
y="0">
                  <Image id="cancelButton" src="casestudy/okay.png"
x="0" y="0" width="89" height="36" />
            </HoverHighlight>
      </buttons>
</confirmationDialog>
```

If you look at the HoverHighlight widget, you'll notice the outOpacity and overOpacity parameters, as was pointed out earlier in the case study. This will be important later when creating the script.

First we need to define the widget:

```
class "HoverHighlight" (Widget)
```

This creates the link to the C++ API and the layout file so MiniUI knows how to parse *HoverHighlight* widgets. In the background, MiniUI is actually creating a linkage in the C++ system to create an instance of this class instead of the generic Widget C++ class. Normally, whenever MiniUI encounters a widget tag, it creates an instance of the standard Widget class. In this case, however, it will create an instance of the Lua class instead.

```
-------------------------------------------------------------------
function HoverHighlight:__init()
-------------------------------------------------------------------
      super();
end
```

This is the constructor. Every widget needs to call the *super()* method inside their constructor. This constructor handles all the needed backend work to link between the C++ API and the Lua class.

```
-------------------------------------------------------------------
```

```
function HoverHighlight:OnLayout( )
-------------------------------------------------------------------
      self.highlightWidget = self:GetChildWidget(0,0);
      self.animator = Animator ( );
end
```

During the OnLayout method, we create the private members and animator object. We'll use the highlight widget object next.

```
-------------------------------------------------------------------
function HoverHighlight:OnLoad(skinElement, widgetElement)
-------------------------------------------------------------------
      self.overOpacity = xpath.ToNumber ( widgetElement, "@overOpacity"
);
      self.outOpacity = xpath.ToNumber ( widgetElement, "@outOpacity"
);
      self.highlightWidget.opacity = self.outOpacity;
end
```

This method loads in the custom parameters needed to animate the highlightable area. As shown, the method extracts the two parameters (over and out opacities) from the widget element in the layout file and stores them for future use. It also sets the opacity of the child widget to the out opacity to initialize it.

```
-------------------------------------------------------------------
function HoverHighlight:Update( timestep )
-------------------------------------------------------------------
      self.animator:Run ( timestep );
end

-------------------------------------------------------------------
function HoverHighlight:OnMouseOver ( x, y )
-------------------------------------------------------------------
      self.animator:Add ( Animate ( self.highlightWidget, "opacity",
"sineInOut", self.overOpacity, 500 ) );
end

-------------------------------------------------------------------
function HoverHighlight:OnMouseOut ( )
-------------------------------------------------------------------
      self.animator:Add ( Animate ( self.highlightWidget, "opacity",
"sineInOut", self.outOpacity, 500 ) );
end
```

This is where the heart of the widget lies; the animation script. What happens is the update method is called every frame. Here, the script updates the animation object with the current timestep. Since MiniUI handles everything in the animation object, no more work needs to be done.

When the user hovers his or her mouse over the widget, the object should animate into one of the two opacity states. When the mouse is over the widget in question, the opacity of that widget should change to the over opacity defined in the layout file. Vice versa, the widget should change to the out opacity in any other case. Here we add the animation objects to the animator. Both have a sine in and out animation to do a smooth translation between the states. As seen, the *opacity* attribute of the highlight widget is updated in this case. Each animation is set to run over the next five hundred milliseconds. Since the

update method runs the animator, no other work needs to be done. On the next update call, the animator will start to run one of these animations.

There's only one thing left to accomplish in this script, events. Now it's possible to set the child widget to fire events whenever an event needs firing, but since highlighting areas are always clickable in our example, I simplified the script to fire from the highlight area.

```
-------------------------------------------------------------------
function HoverHighlight:OnMouseDown ( x, y )
-------------------------------------------------------------------
      self.highlightWidget:Fire ( "OnClick", { } );
end
```

Normally a widget will run its own fire method, but we want the event to be received from the child widget instead. In our case, the buttons themselves will fire the event so as not to confuse the developer.

With this final script file done, the entire widget can be shown anywhere inside a working MiniUI based user interface application.

Breaking down the C++ API

MiniUI's C++ API allows the developer to communicate directly with the widgets defined in the XML and script files. It also is required to setup MiniUI properly to be run.

IArchive class

IArchive
+~IArchive() *+GetArchiveName() : string* *+LoadFile(in filename : string) : istream ** *+FileExists(in filename : string) : bool*

Although the *IArchive* class is a simple class, it defines one of the basic functionalities needed by the entire user interface system. It defines access to the file system dependent on the base hardware and software architecture the MiniUI system runs on.

MiniUI has no concept on what a file system is. Instead, it offloads this to the application. Although it may seem that a simple C++ based file system accessor would be the proper method, it lacks in one key area. A C++ based file system accessor limits the developer to simple files in a directory. It does not allow access inside an archive such as a tar or zip file.

The IArchive instead abstracts the main functionality needed to load a file (script, XML, image) into the IArchive class. This allows the developer to add in custom loaders for archives the application needs. This can save space and decrease load times for large quantities of files.

The example application in the MiniUI system shows how to create a simple file loader using the *IArchive* class. It is up to the reader to look at this example since it's outside the realm of this document. Instead, the core *IArchive* members will be defined. It can be found at:
/src/Integration/FileSystem/FolderArchive.h.

string IArchive::GetArchiveName( ) [overridden]

This method simply returns an identifier for the archive being loaded. It allows the developer to access the contents of the archive through a simple path instead of loading directly from the file. Mostly, this method returns the path to the archive or directory being loaded from.

For example, if this method returned **/usr/local/miniui.zip**, the developer can load a file from the archive using the path **/usr/local/miniui.zip/innerpath/myimage.png**. This makes it much simpler to load from an archive.

Also, if the path given is not absolute, as shown above, the archive system will search through all open archives for the file. For example, if the path was **innerpath/myimage.png**, the archive system will still find the the file in miniui.zip.

This is a powerful system for loading files. It also speeds up development by not having to archive all the files during each build. Instead, a folder based *IArchive* object can be created to load the files, and only during release, use the zip archive. By changing one line of code, the developer can use a completely different archive.

One last major point here includes adding extensibility. If the developer wants to allow user defined skins, but not change some key elements (such as the company logo), a custom, encrypted archive can be used for those non changeable elements, and an open archive (such as zip) for changeable ones.

istream* IArchive::LoadFile(string filename) [overridden]

LoadFile takes in a relative path to a file and returns the istream object that defines the file. All file accessors are standard *istream* objects as defined in the C++ standard template library. The *istream* class is already an abstract accessor into any time of stream (file, string, etc.).

If the file cannot be loaded, or the file does not exist at the path given, LoadFile must return a NULL value to denote this.

bool IArchive::FileExists (string filename) [overridden]

To speed up processing of the filesystem, MiniUI makes use of the FileExists method to detemine which archive a relatived path file exists. Simply, FileExists returns true if the file at the filename exists, and false otherwise.

Mouse class

«struct»Input::**Mouse**
+x : int +y : int +leftMouseDown : bool +rightMouseDown : bool
+Mouse()

The *Mouse* class contains all the necessary information about the pointing device that is needed by the MiniUI library. This structure must be updated every frame and sent in to the MiniUI system. This allows the MiniUI framework to handle pointer based events autonomously.

Please take note that the *Mouse* class may contain information about any similar pointing device and not limited to an actual computer style mouse. It can be replaced with information from a stylus or finger on a touch screen. It can also be from a tablet or

optical scanning device. The *Mouse* class is a simple abstraction for any style of pointing device.

This was done, obviously, to insure that the MiniUI library could be ported to portable electronics (PDAs, cell phones, etc.) that are unencumbered by a large mouse.

UIManager class

MiniUI::**UIManager**
+OnUIEvent : UIEvent -_luaVM : LuaVirtualMachine -_skin : Skin -_pCurrentScreen : Screen * -_screens : ScreenMap -_textureManager : TextureManager
+UIManager() +~UIManager() +LoadUIArchive(in pArchive : IArchive*) : bool +LoadScreens(in rootXML : string, in pArchive : IArchive*) : bool +Initialize() : bool +Update(in timestep : unsigned int, in pMouse : Mouse*) : bool +SetCurrentScreen(in name : string) +GetCurrentScreen() : Screen * +FireEvent(in pWidget : Widget*, in event : string, in o : object &) -InitializeScripts() : bool -Register() -UpdateWidget(in pWidgetList : WidgetList*, in timestep : int, in pMouse : Mouse*)

The *UIManager* class contains the core functionality of the MiniUI system. All calls between the user interface and the application business code will be run through this class. Because of it's global need in the application, the *UIManager* class follows the singleton design pattern. However, the developer must make sure he or she creates an instance of the object before any access to the singleton object is made.

The *UIManager* holds all the pieces that are a part of the entire MiniUI library, such as the Lua interpreter, XML parsers, screen list, and graphical texture manager. These components are hidden away from the developer, but some extensions can be made as described later.

Because of the extreme size of the MiniUI system, only the publicly accessible methods methods are defined here. To get information on the private/protected methods, please reference the MiniUI API Reference Manual.

UIManager::UIManager( )

The constructor simply creates an instance of the UI manager object.

bool UIManager::Initialize( )

Initialize runs the initialization of the entire MiniUI system. It's the first method to run on the *UIManager*. Any call run before Initialize will have undefined behavior.

bool UIManager::LoadUIArchive (IArchive *)

This method takes an *IArchive* object and stores it into the archive manager. It also loads all the files from this archive used in the user interface into memory. This includes loading and parsing the skin XML files and loading any external dependencies these files may have. It returns true if the files were parsed properly and everything was loaded.

The LoadUIArchive method looks for a **skin.xml** file to load in the root path of the archive. This is the starting point of the entire user interface.

bool UIManager::LoadScreens (string filename, IArchive *)

After the skin file has been loaded from the archive, the screens need to be loaded. This method takes in the root XML file for the screens and loads up the XMLs using the predefined skin.

Once this method completes, the user interface is fully loaded and can then be used in the application. All references to screens and widgets in the user interface are created in C++ and can be used at any time from this point on.

bool UIManager::Update (int timestep, Mouse*)

This method is the heart of the MiniUI user interface library. It draws the individual frames of the user interface, handles all animations, and even handles the event system. The developer must run this method every frame, and must send in the time (in milliseconds) since the last frame.

The mouse information is also sent in. This allows MiniUI to handle any mouse/finger events used in the script files.

void UIManager::SetCurrentScreen (string screenName)

SetCurrentScreen will change the screen being shown to the one that matches the screen name passed. The elements inside of the new screen are instantly available to the C++ and script code.

On the next update, MiniUI will draw the first frame of the screen. Note that the update will contain the time since the last frame, and not the time since the screen was last shown.

Screen* UIManager::GetCurrentScreen ( )

This method simply gets the current screen being shown. If no screen is currently shown, NULL is returned.

void UIManager::FireEvent (Widget* w, string name, object o)

This method fires an event off in the application space. It creates an event object and fires off UIManager::OnUIEvent. The event takes the name of *name* and uses *o* as the parameters.

This is usually fired off from the script using *Widget:Fire()*. It in turn will run the UIManager::OnUIEvent event, which is described in the Event System section.

Screen class

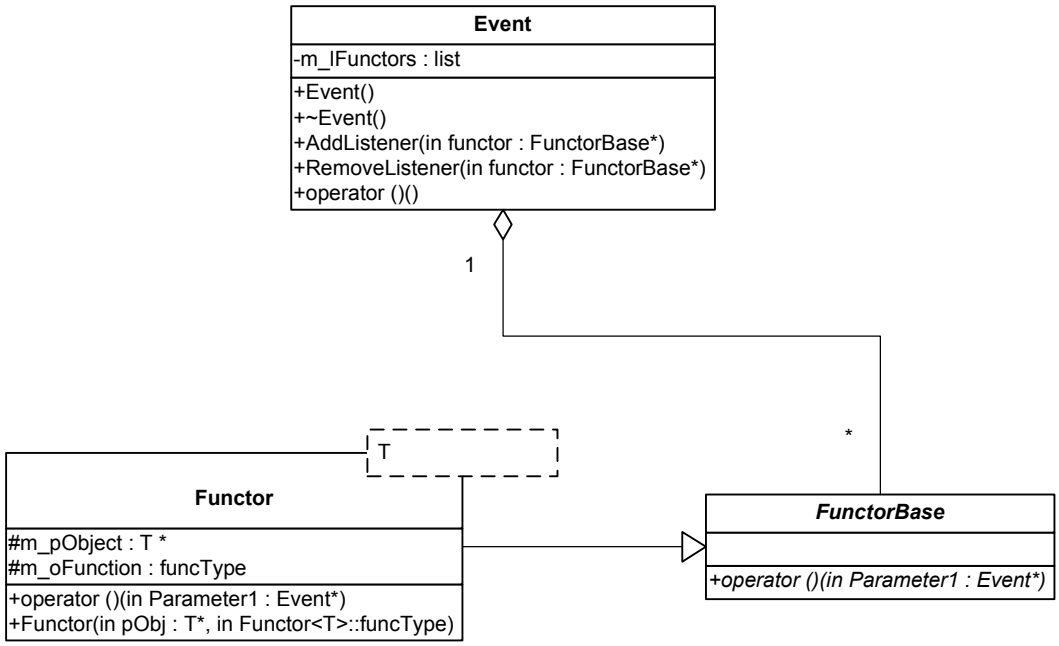
Screen
-_id : string -_pElement : TiXmlElement * -_widgetList : WidgetList
+Screen() +~Screen() +LoadScreen(in pElement : TiXmlElement*) : bool +BuildScreen(in pSkin : Skin*) : bool +id() : string +GetWidgetList() : WidgetList * +FindWidget(in id : string) : Widget * #FindWidgetChild(in id : string, in pWidget : Widget*) : Widget * #LoadWidget(in pChild : TiXmlElement*, in pSkin : Skin*, in depth : int = 0) : Widget * #LoadLayout(in pRenderable : Renderable*, in pChild : TiXmlElement*, in pWidget : TiXmlElement*) : bool #LoadChildren(in pWidget : Widget*, in pChild : TiXmlElement*, in pWidgetEl : TiXmlElement*, in pSkin : Skin*, in depth : int) : bool #OnLayout(in pWidgetList : WidgetList*)

The *Screen* class contains information of a screen that can be shown on the user interface. It stores all the widgets of the screen in a hierarchical structure. In the business logic of the C++ code, the developer will have to use widgets directly to make calls.

Widget* Screen::FindWidget(string id)

This method simply takes in the id of a widget and finds it in the hierarchy of widgets. If the widget, however, is not found, it will return a NULL value. This method only returns the first widget based on the ID found. It is because of this limitation that ids should be unique across an entire screen. Ids across screens can be similar though, since they never share the same namespace.

The Event System



The event system is a simple system to handle any type of event that may occur in the application. MiniUI, however, only uses it for user interface type events that the script may fire. The obvious choice to registering event callbacks is to use a C++ based callback system.

C++ has basic limitations around callbacks. First, it cannot make a callback to a member method, only to static. Secondly, only one method can be added to a callback, even though multiple classes may need access to the information.

To get around this, MiniUI has its own event system that caters to both of those issues. First, the developer can now have callbacks to member methods. This makes sure that the application code can stay object oriented without the use of globals for accessing member functions.

Second, an event can have many different consumers. This allows a lot of freedom in the way events are used by not limiting the applications use of them. Instead, if an object needs access to an event, it can register for it without worrying about whether it's taking over access from another object.

When a system is finished with an event, it is expected for that object to unregister for the event. If it does not, and the object is deleted, unknown results can occur.

Event Class

Event
-m_lFunctors : list
+Event() +~Event() +AddListener(in functor : FunctorBase*) +RemoveListener(in functor : FunctorBase*) +operator ()()

The event class is the base class for all the generic events used in MiniUI (or in the application). It stores all the callbacks needed to call during an event, and cleans up afterwards.

To actually create a custom event, the developer will need to subclass the *Event* class with the valid public accessors needed by the callbacks.

Event::Event( )

The constructor simply creates an instance of the event object.

Event::~Event( )

The destructor deletes all the object's members and unregisters all registered callbacks.

void Event::AddListener (Functor* functor)

This method takes in a function object (functor) and adds it to the list of listeners. When the event is fired, it calls each listener in the order they were added.

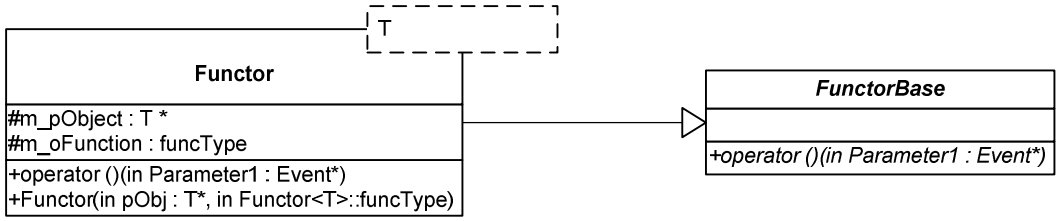
void Event::RemoveListener (Functor* functor)

This method removes a function object from the list of listeners. If the functor is not actually listening at that point, nothing happens.

void Event::operator () ( )

This operator fires off the event. An operator was used instead of a method call because the event should, in the end, look like a functor. For example, if the developer has an *OnClick* event, he or she could create the *OnClick* object and run *OnClick( )*. This makes it look like the application is calling an *OnClick* event.

The Functor Class



A functor is a simple object oriented callback mechanism. Normal callbacks cannot callback to member functions unless they are static. To actually call a member function, the inherit *this* pointer must be known and passed in.

The *Functor* class stores the self referencing pointer and method pointer in a single object. This way, one simple method call will handle the complex dereferencing of all the pointers.

Callbacks take only one form to simplify the way we use events. They all return void and take in a pointer to an Event. It is up to the callback to determine the event type and run a static cast on that pointer to the correct subtype.

The format for the callback is of the following:

```
void ClassName::OnGenericEvent ( Event* pEvent );
```

Functor::Functor<T>(T* object, FuncType* functionPointer)

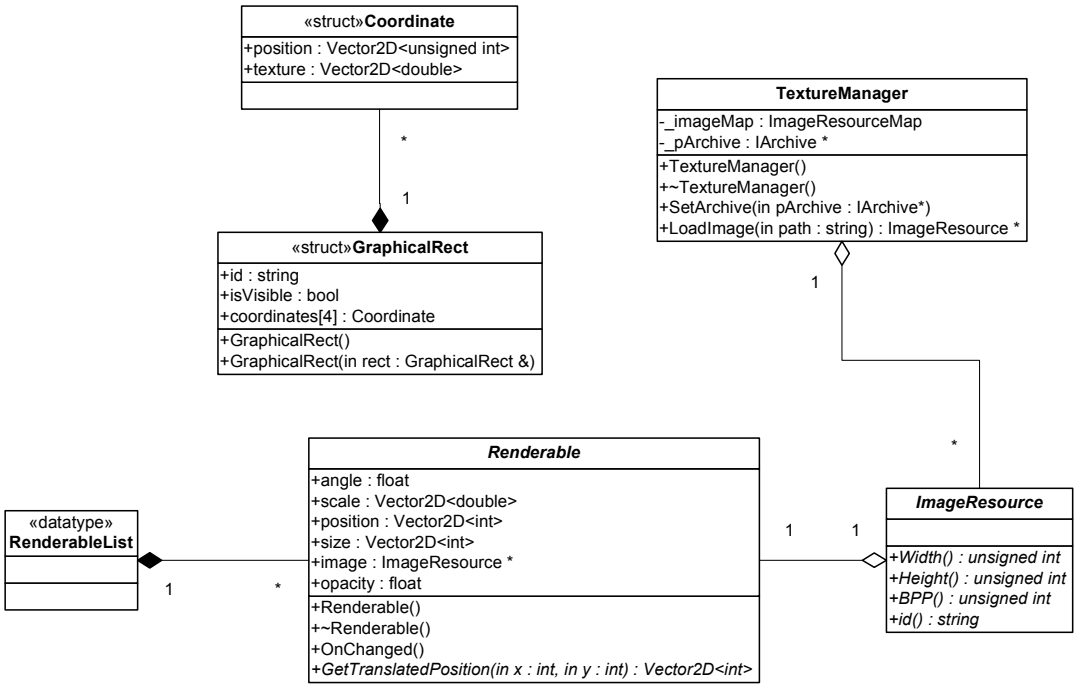
This constructor creates a functor object to pass to an *Event* object. The object is a pointer to the object that will have the method call, while functionPointer is a pointer to the method as if it were a static method.

C++ object methods have an inherit class pointer that is passed in before other parameters. This is the only difference between object methods and standard functions. This constructor handles this difference automatically by sending in the pointer to the object initially.

void Functor::operator() (Event* pEvent)

This operator calls the callback with the stored pointers and the event object.

The Graphics System



MiniUI makes no assumption of the graphical environment running on the current system. You can't make an assumption of this without sacrificing cross compatibility. For example, a Windows computer may be running under basic Win32 methods or up to Direct3D 10.0. On the other hand, an embedded device may be using a software renderer or OpenGL ES.

MiniUI was built from the ground up with hardware acceleration in mind. It makes certain assumptions of the underlying environment to handle these things, such as caching drawing commands, handling widget scaling and rotating, and defining widget space versus screen space.

MiniUI has an API for developing an external library to handle these tasks. In the attached example program, an OpenGL ES 1.0 compatible library is used. It is up to the reader to go through this code separately as a reference since the implementation is outside the realm of the base MiniUI project.

The MiniUI graphics layer API is made up of only a few distinct classes. The IRenderer interface defines how MiniUI interacts with the graphics layer. The Renderable class defines the drawn widget in a graphics API specific way. Lastly, the ImageResource class defines the image used to draw the widget. This is usually defined as a “texture” in most graphics systems.

IRenderer class

IRenderer
+Update(in pScreen : Screen, in pMouse : Mouse*)* *+CreateRenderable() : Renderable **</br>

The IRenderer class defines the access layer into the rendering system used by the application. MiniUI will make a call to this class every frame as needed to make sure the graphics are drawn properly. It will first move and animate all widgets before making this call, however, to simplify the graphics layer's task to only drawing.

void IRenderer::Update (Screen* screen, Mouse* mouse)

This method is called ever frame to draw the screen passed in. It is up to the graphics layer to use this information to finally draw all the shown widgets in the passed in screen.

The mouse parameter, however, is used differently. The mouse values passed in are raw screen values and need to be translated into widget space. Since many graphics APIs can determine widget space (based on the actual matrices used to define the widget space) using hardware acceleration. The *Update* method should determine the widget space of the widget and pass these updated values to the widget during the redraw.

Renderable* IRenderer::CreateRenderable ( )

This method is called to create a new *Renderable* object using in the application. This a simple factory interface to create custom *Renderable* objects based on the application's specific graphics requirements. This method should simply return a new instance of the custom *Renderable* class based on the graphics API.

Renderable class

Renderable
+angle : float +scale : Vector2D<double> +position : Vector2D<int> +size : Vector2D<int> +image : ImageResource * +opacity : float
+Renderable() +~Renderable() +OnChanged() *+GetTranslatedPosition(in x : int, in y : int) : Vector2D<int>*

The *Renderable* class defines the graphical layout and matrix math for a widget. It contains a list of the *GraphicalRect*s used to draw this widget. It also contains the current angle, scale, and position of the widget itself.

It is up to the *IRenderer* class to convert the parameters used in this class into drawing commands. Usually the *Renderable* class will contain the most graphics API specific code out of all the classes. Its job is to cache any drawing commands, handle the

conversion of screen space to widget space, and store the raw formatted graphics information.

Renderable::Renderable( )

The constructor simply creates an instance of the abstract *Renderable* object.

Renderable::~Renderable( )

The destructor deletes all the object's members.

void Renderable::OnChanged ( )

This method is where the caching of the drawing commands hides. When initially created, the widget will have its *OnChanged* method called to store any drawing commands. It will also be called whenever the widget's drawing commands may need to change due to changing of the underlying structure of the widget.

Vector2D<int> Renderable::GetTranslatedPosition (int x, int y)

As described before, the *Renderable* class has the responsibility of translating a point from screen space into widget space. This is inherently due to hardware accelerated graphics APIs usually have space translation built into its specifications. This simplifies the need to do complex and redundant matrix mathematics which the graphics processor has already done.

A screen space position is passed in, and the translated position is returned. Note that when a screen space does not match with the widget positioning and size, a negative position may be returned. A position outside of the widget area can also be returned. The developer must handle this properly since some widgets will get position updates of the pointing device even though the pointer is outside the widget space area.

ImageResource class

ImageResource
+Width() : unsigned int *+Height() : unsigned int* *+BPP() : unsigned int* *+id() : string*

The *ImageResource* class defines the information needed by MiniUI of an image or graphics texture. In a software renderer, it will contain all the information needed to store an image in memory for easy drawing, sometimes many times a frame.

On a hardware accelerated platform, the graphics processor stores the image internally in its own memory and usually returns a pointer or index to the image internally. These images are normally called textures for hardware accelerated graphics. Therefore, the *ImageResource* object should keep the index or pointer only and the other useful pieces of information.

unsigned int ImageResource::Width ( )

Returns the width, in pixels, of an image resource.

unsigned int ImageResource::Height ( )

Returns the height, in pixels, of an image resource.

unsigned int ImageResource::BPP ( )

Returns the number of bits used to defined each pixel. This value must be divisible by eight to be valid.

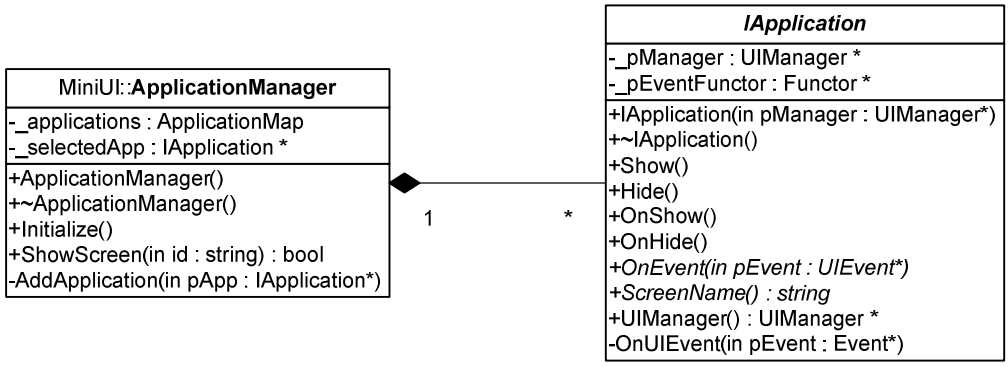
string ImageResource::id ( )

Returns the unique identifier for this resource. This is used internally to make sure that no more than one copy of an image resource is loaded in memory. To simplify this task, *id()* usually returns the path to the image on the disk.

Tying It Together

So, the three steps to designing a user interface have been defined, but that still misses the entire picture of tying it all together. At this point I want to discuss the example application included with the source code, since it's a good example of how an application should be created.

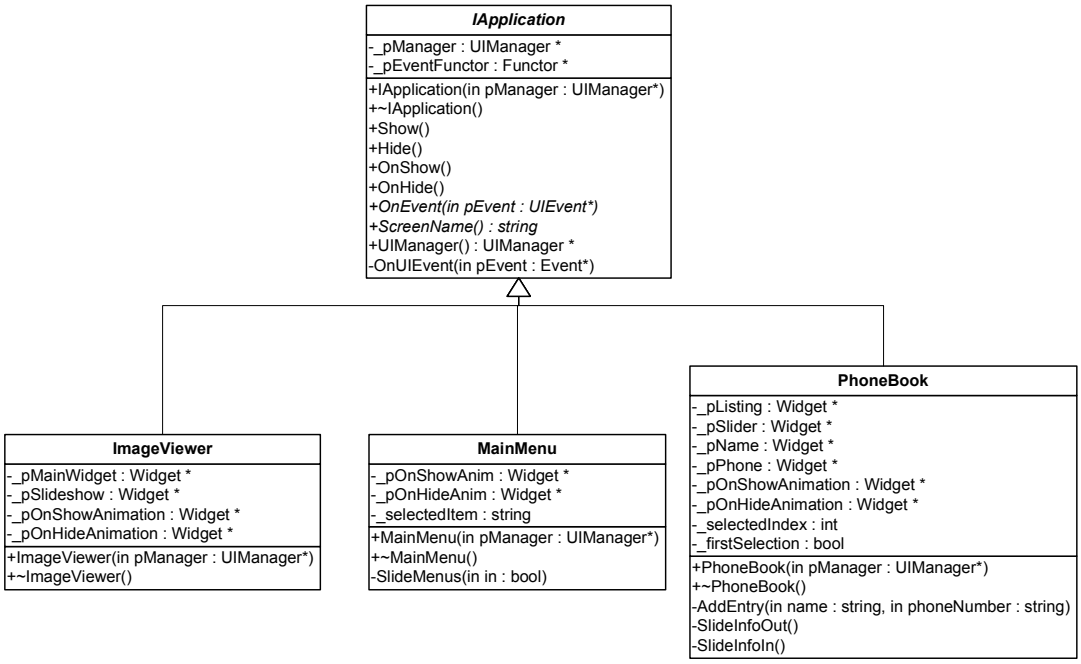
I won't go into major details on how the application is put together since it is well documented within the source code. Instead I'll discuss the design of the example application. It may be of some help when designing future applications using the MiniUI framework.



In this example, I have an application manager that contains all the applications that can run on this system. As seen here, the application manager holds the shown application in memory to pass events to it. In actuality, when calling *ShowScreen()*, the application manager registers the *OnEvent()* for that class with MiniUI to notify it of user interface events. If a previous screen was shown, it's *OnEvent()* method would be deregistered with MiniUI. This makes sure that only the top running application is handling user interface events.

Of course, it's shown that the applications are run as event driven, similar to many other user interface applications. The applications can run code during the OnShow and OnHide events, and also during script events defined in the Lua scripts. It is up to future applications to handle running threads if needed for other background tasks. This is a common way of setting up applications in many graphical environments, such as Microsoft's MFC.

As shown next, there are three different applications that can run.



As seen here, there are three applications that run in the example. First there is the main menu which controls running the other two applications. In this screen, there are two buttons to press, one for each new application. When the button is pressed, it fires an event to the MainMenu class which makes a call to the ApplicationManager to load the new screen.

The ImageViewer is a simple application for displaying a list of images. Each image can be selected by hitting either the left or right arrow. When one of these buttons is hit, it fires an even to the ImageViewer class. When this event is found, the class runs whatever animation is defined (in this case, a fade) and loads up the next image.

Lastly the PhoneBook application shows how it's simple to dynamically add data to a screen using the C++ API. Basically, all that is done is passing a simple Lua object to a method inside the scripts. It is easy to see in the example code how this is performed. This is also shown in the LuaBind documentation defined earlier.

As an example, the code should be enough to show how it's possible to tie the C++ code into the MiniUI library. A lot of effort has been placed into the library to simplify usage of MiniUI.

Conclusion

Graphical user interfaces are common place in today's world of mobile electronics and personal computers. Applications are more complex and take more time to develop than ever before. Of course, as more applications become user interface heavy, the stranglehold code developers have on the user interface development can cause strain on the application's schedule.

By offloading 100% of the user interface design to user interface designers, it allows the application to be done in a faster time frame with less need for programmers to worry about how the user interface will work or interact.

As shown in the example program, the applications become simple event driven pieces. To perform many of the tasks that MiniUI performs, such as the animations, frame by frame manipulation of the widgets must be performed. Also the application code would need to know what the exact widgets were to handle some of the more advanced manipulations. Of course, MiniUI allows the designers to offload all of this advanced work into the skin, layout, and script files. This means that a full user interface can be developed outside of the application code, and can handle all of the advanced layout and animations seen in high end electronics, such as Apple's new IPhone.

Also, a nice side effect of using the library is that the user interface becomes completely portable to another operating system or even another hardware platform with zero code changes. As a hardware agnostic system, once MiniUI is compiled for a different platform, the user interface becomes completely portable.

References

The Joy of XUL. 9 Sep 2007. Mozilla.
<http://developer.mozilla.org/en/docs/The_Joy_of_XUL>

Shriener, Dave et al. OpenGL Programming Guide. Addison-Wesley Professional, 2004. < http://glprogramming.com/red/>

McConnell, Steve. Code Complete: Second Edition. Microsoft Press, 2004.

Gamma, Erich et al. Design Patterns. Addison-Wesley, 1995.

Appendix A – Skin DTD

```
<!--
<!DOCTYPE skin[
<!ELEMENT skin (widgets+,fonts+)>
<!ELEMENT widgets (widget+)>
<!ELEMENT widget (layout,childArea,eventHandler+)>
<!ELEMENT layout (section+)>
<!ELEMENT section (source,destination)>
<!ELEMENT eventHandler (event+)>
<!ELEMENT event (parameter+)>
<!ELEMENT fonts (font+)>

<!ATTLIST widget id CDATA #REQUIRED>
<!ATTLIST widget src CDATA #REQUIRED>
<!ATTLIST widget width CDATA #REQUIRED>
<!ATTLIST widget height CDATA #REQUIRED>

<!ATTLIST section id CDATA #REQUIRED>

<!ATTLIST source x CDATA #REQUIRED>
<!ATTLIST source y CDATA #REQUIRED>
<!ATTLIST source width CDATA #REQUIRED>
<!ATTLIST source height CDATA #REQUIRED>

<!ATTLIST destination x CDATA #REQUIRED>
<!ATTLIST destination y CDATA #REQUIRED>
<!ATTLIST destination width CDATA #REQUIRED>
<!ATTLIST destination height CDATA #REQUIRED>

<!ATTLIST childArea select CDATA #REQUIRED>
<!ATTLIST childArea x CDATA #REQUIRED>
<!ATTLIST childArea y CDATA #REQUIRED>
<!ATTLIST childArea width CDATA #REQUIRED>
<!ATTLIST childArea height CDATA #REQUIRED>

<!ATTLIST area id CDATA #REQUIRED>
<!ATTLIST area x CDATA #REQUIRED>
<!ATTLIST area y CDATA #REQUIRED>
<!ATTLIST area width CDATA #REQUIRED>
<!ATTLIST area height CDATA #REQUIRED>

<!ATTLIST eventHandler class CDATA #REQUIRED>

<!ATTLIST event handler CDATA #REQUIRED>
<!ATTLIST event type CDATA #REQUIRED>
<!ATTLIST event eventArea CDATA #REQUIRED>
<!ATTLIST event data CDATA #REQUIRED>

<!ATTLIST parameter id CDATA #REQUIRED>
```

```
<!ATTLIST font id CDATA #REQUIRED>
<!ATTLIST font src CDATA #REQUIRED>
]>
```

Appendix B – Project Dependencies

MiniUI requires a few external projects to function. Some are included by default in the code (where the license allows it), while some are dynamically linked or output used.

Library Dependencies

- TinyXML – XML Parsing library
- TinyXPath – XPath query library
- Lua – Scripting language
- Luabind – C++ and Lua binding library

Example Application Dependencies

- SDL – Graphics layer
- OpenGL – 3D graphics layer

Build Dependencies

- Linux operating system – Kernel 2.6
- scons – Build system
- KDevelop – Integrated development environment